

Contents

Introduction	2
1. BASICS OF SOFTWARE ENGINEERING	3
2. Types of software Engineering:.....	4
1. Classical software engineering.....	4
Water flow model:	4
Spiral method:.....	4
V- model.....	5
2. Agile software engineering:	6
Scrum methodology:.....	6
Kanban method:.....	6
Extreme Programming (XP).....	6
3. Operating System:	7
Basic functions of Operating system:	7
4. DIFFERENT DevOps tools:.....	8
1.Pylint	8
2.PyUnit.....	10
3.Git/Git hub:	10
4.DOCKER/DOCKER HUB:	23
5.Kubernetes	35
Kubernetes Architecture:	35
6.ANSIBLE	40
Summary:	43

Authors:

N Sukhitha Bhushan

1RVU23CSE293

Section B

BTech(hons). Computer Science

RV University.

Introduction

The document provides a thorough exploration of software engineering principles, methodologies, operating systems, and essential DevOps tools. It begins by detailing classical software engineering approaches such as Waterfall, Spiral, and V-Model, contrasting them with Agile methodologies like Scrum, Kanban, and Extreme Programming (XP), each tailored for iterative development and continuous improvement. The role of operating systems in managing hardware and software resources is highlighted, covering critical functions such as file, memory, and process management. Additionally, the document dives into key DevOps tools like Git, Docker, Kubernetes, and Ansible, essential for automation, collaboration, and efficient software deployment in modern development environments.

Authors:

N Sukhitha Bhushan

1RVU23CSE293

Section B

BTech(hons). Computer Science

RV University.

DEVOPS- tools used in software engineering

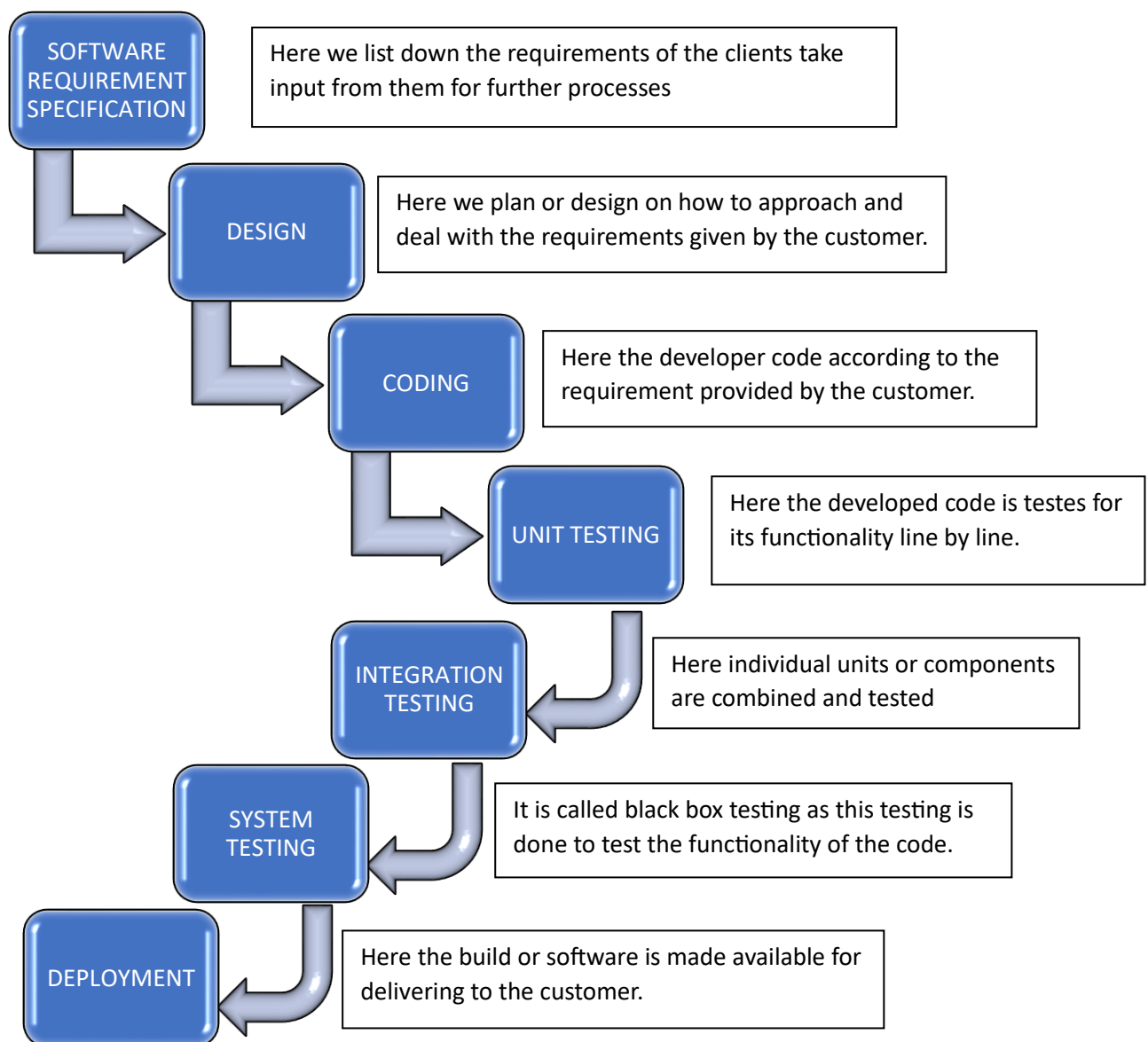
1. BASICS OF SOFTWARE ENGINEERING:

DevOps- it is an approach the modern software engineering community uses for collaboration, automation, continuous build, continuous integration and deployment.

Dev - Ops which stands for,

Dev = developers and Ops = operations.

Software development life cycle (SDLC):

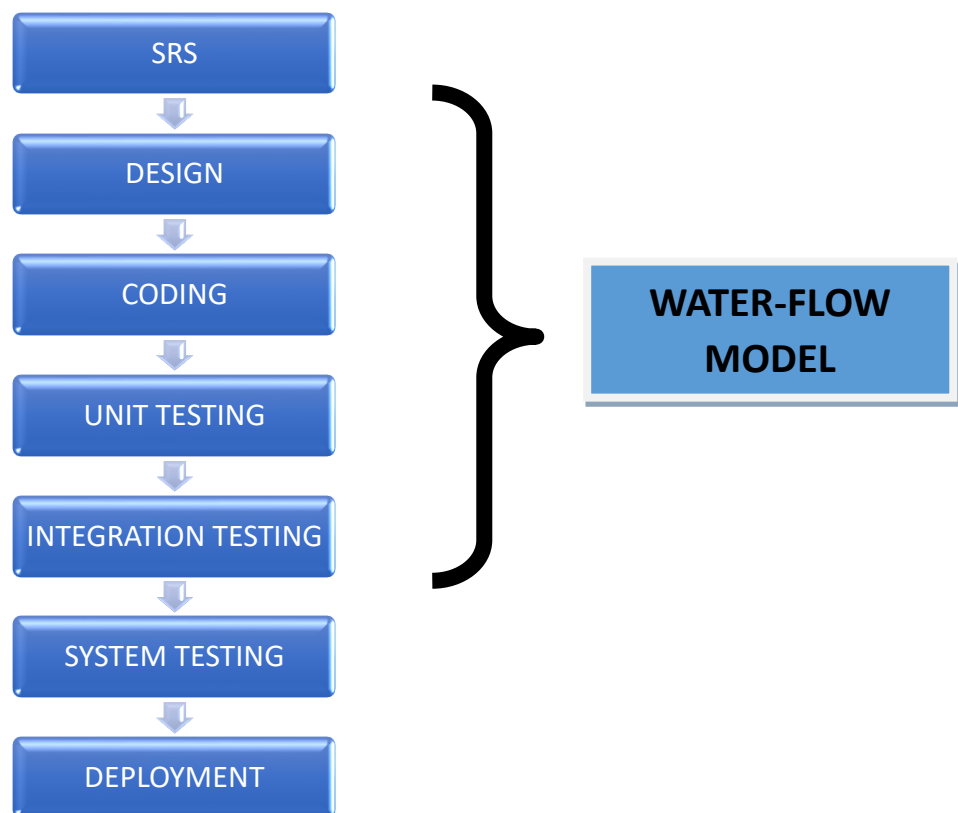


2. Types of software Engineering:

1. Classical software engineering:

Water flow model:

This is a classical approach of developing a software, Waterfall model is a linear and sequential software development methodology where each phase (requirement analysis, design, implementation, testing, deployment, and maintenance) must be completed before the next one begins.



Representation of water-fall model

Spiral method:

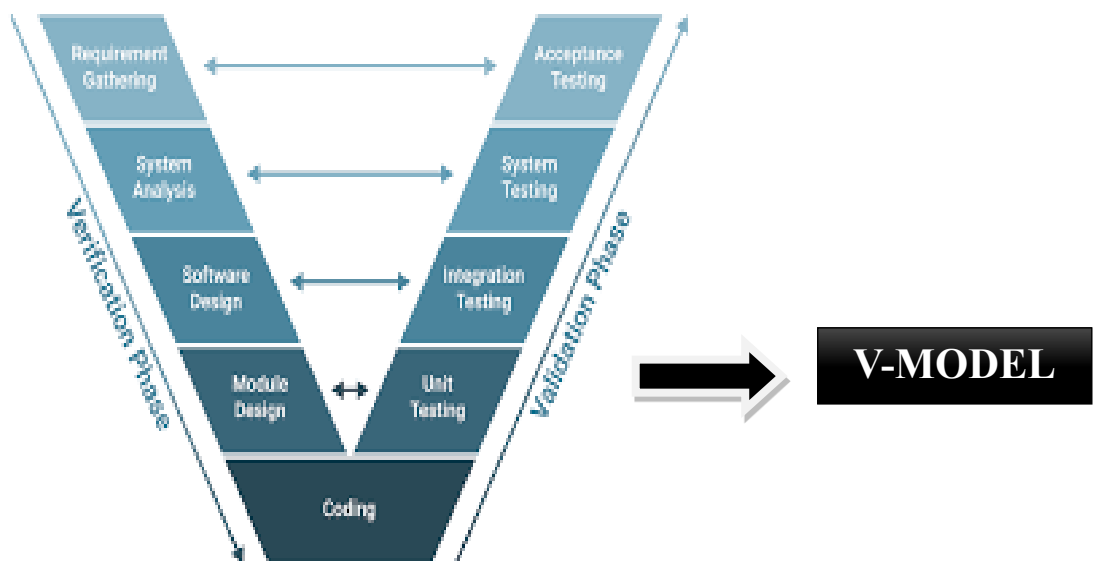
This is a classical model where the review of the customer is taken after each requirement submission, this method combines iterative development (prototyping) and systematic aspects of the Waterfall model, emphasizing risk analysis and refinement through repeated cycles (spirals) of planning, risk assessment, engineering, and evaluation.



Representation of spiral model

V- model

The V-Model (Validation and Verification model) is a software development methodology where development and testing activities are planned in parallel, forming a V shape. Each development phase (requirements, design, implementation) has a corresponding testing phase (acceptance testing, system testing, integration testing, unit testing) to ensure validation and verification at every stage.

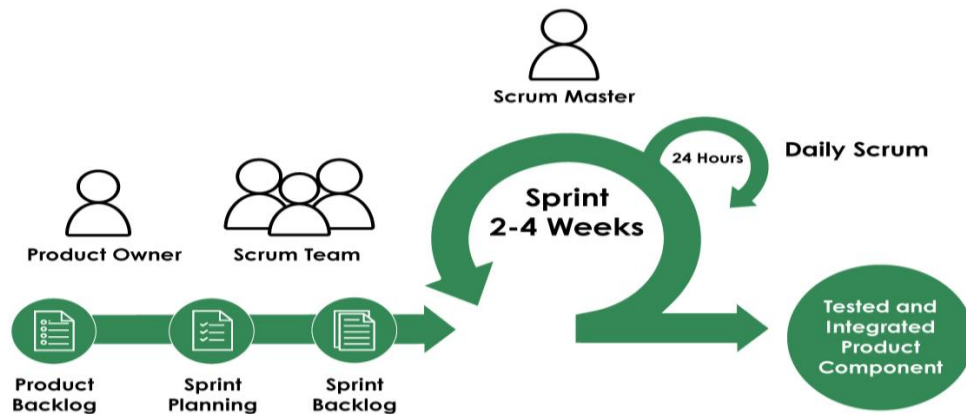


Visual Representation of v-model

2. Agile software engineering:

Scrum methodology:

The Scrum method is an Agile framework for managing and completing complex projects. This is an iterative progress through small, time-boxed sprints and involves several key roles, artifacts, and ceremonies. It breaks projects into small, time-boxed iterations known as sprints, typically lasting 2-4 weeks, to deliver incremental improvements. Key roles in Scrum include the Scrum Master, who ensures the team follows Scrum practices; the Product Owner, who manages the product backlog and prioritizes tasks; and the Development Team, which executes the work. Scrum emphasizes regular meetings such as daily stand-ups, sprint planning, sprint reviews, and retrospectives to maintain transparency, encourage continuous feedback, and adapt to changes swiftly.



Kanban method:

Kanban is a visual project management method that helps teams manage workflow efficiently by visualizing tasks on a board. Originating from lean manufacturing, it uses cards and columns to represent tasks and their stages in the process, promoting continuous delivery and improvement. The method emphasizes limiting work in progress to enhance focus and productivity. It fosters collaboration and flexibility, making it easier to adapt to changes and optimize workflow.

Extreme Programming (XP)

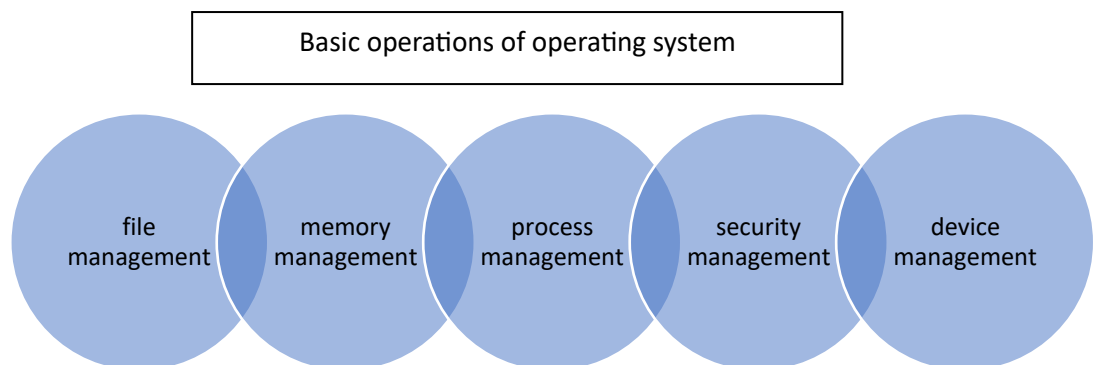
It is an agile software development methodology that emphasizes customer satisfaction, flexibility, and teamwork. It advocates frequent releases in short development cycles to improve productivity and introduce checkpoints where new customer requirements can be adopted. Key practices of XP include pair programming, test-driven development (TDD), continuous integration, and collective code ownership. XP focuses on enhancing software quality and responsiveness to changing customer needs through a set of core values: simplicity, communication, feedback, courage, and respect.

3. Operating System:

An operating device (OS) is the essential software program that manages laptop hardware and software program resources, imparting a strong and constant environment for packages to run. It handles important responsibilities such as memory control, system scheduling, enter/output operations, and document gadget management. The OS acts as an intermediary between customers and the pc hardware, permitting green execution of applications and effective consumer interplay. Popular working systems include Windows, macOS, Linux, and Android, every providing unique features and talents tailored to special kinds of gadgets and consumer needs.

Basic functions of Operating system:

1. File management: The OS manages the creation, deletion, and access to files and directories. It provides file systems that allow programs to access files using a common interface, regardless of the physical storage device used
2. Memory management: The OS manages the allocation and deallocation of memory to processes. It provides virtual memory to allow programs to run even if the available physical memory is not sufficient.
3. Device management: The OS manages the interaction between the computer hardware and the software. It provides device drivers that allow applications to communicate with the hardware devices such as printers, keyboards, and monitors.
4. Security management: The OS provides a secure environment for the user and applications. It enforces access control policies to restrict access to sensitive resources and information.
5. Process management: The OS manages the creation, scheduling, and termination of processes. It allocates resources such as CPU, memory, and I/O devices to each process and provides inter-process communication mechanisms for the processes to share data and synchronize their activities.



4. DIFFERENT DevOps tools:

1.Pylint:

this is a tool which checks a python file for its proper commenting so that the code is understandable to the user of the code. Pylint is your smart friend who checks your spelling, grammar, and makes sure your story makes sense. In the world of coding, Pylint checks your code for mistakes and gives you suggestions to make it better. It helps coders write clean, error-free programs.

Pylint tool is similar to **pychecker**, **pyflakes**, **flake8**, and **mypy**.

INSTALLATION:

To install pylint, make sure Python is installed on your PC. Open the command prompt (Windows) / terminal (Linux) on your PC and type the following command

```
pip install pylint
```

```
C:\Users\krubhuu_2000>pip install pylint --user
Collecting pylint
  Downloading pylint-3.3.6-py3-none-any.whl.metadata (12 kB)
Collecting astroid<=3.4.0.dev0,>=3.3.8 (from pylint)
  Downloading astroid-3.3.9-py3-none-any.whl.metadata (4.5 kB)
Requirement already satisfied: colorama>=0.4.5 in c:\users\krubhuu_2000\appdata\local\programs\python\python311\lib\site-packages (from pylint) (0.4.6)
Requirement already satisfied: dill>=0.3.6 in c:\users\krubhuu_2000\appdata\local\programs\python\python311\lib\site-packages (from pylint) (0.3.9)
Collecting isort!=5.13,<7,>=4.2.5 (from pylint)
  Downloading isort-6.0.1-py3-none-any.whl.metadata (11 kB)
Collecting mccabe<0.8,>=0.6 (from pylint)
  Downloading mccabe-0.7.0-py2.py3-none-any.whl.metadata (5.0 kB)
Requirement already satisfied: platformdirs>=2.2 in c:\users\krubhuu_2000\appdata\local\programs\python\python311\lib\site-packages (from pylint) (4.3.6)
Requirement already satisfied: tomlikit>=0.10.1 in c:\users\krubhuu_2000\appdata\local\programs\python\python311\lib\site-packages (from pylint) (0.13.2)
Downloading pylint-3.3.6-py3-none-any.whl (522 kB)
Downloading astroid-3.3.9-py3-none-any.whl (275 kB)
Downloading isort-6.0.1-py3-none-any.whl (94 kB)
Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
Installing collected packages: mccabe, isort, astroid, pylint
  WARNING: The scripts isort-identify-imports.exe and isort.exe are installed in 'C:\Users\krubhuu_2000\AppData\Roaming\Python\Python311\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
  WARNING: The scripts pylint-config.exe, pylint.exe, pyreverse.exe and symilar.exe are installed in 'C:\Users\krubhuu_2000\AppData\Roaming\Python\Python311\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed astroid-3.3.9 isort-6.0.1 mccabe-0.7.0 pylint-3.3.6
```

To verify the pylint installation, type the following command

```
pylint --version
```

```
C:\Users\krubhuu_2000>pylint --version
pylint 3.3.6
astroid 3.3.9
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)]
```

You should see pylint “2.4.4” version. We can also verify the installation by reinstalling the pylint. In that case, if pylint is already installed you should see **Requirement already satisfied** on your screen.


```
A = 1
B = 2
print('Sum of Numbers:', A + B)
```

Consider the following program that accepts two numbers and prints their sum.

Now save the above program in the file **add.py**

Open your command prompt / terminal and type the following command

Pylint add.py

```
C:\Users\krubhuu_2000\Desktop>pylint add.py
***** Module add
add.py:3:0: C0304: Final newline missing (missing-final-newline)
add.py:1:0: C0114: Missing module docstring (missing-module-docstring)
add.py:1:0: C0103: Constant name "a" doesn't conform to UPPER_CASE naming style (invalid-name)
add.py:2:0: C0103: Constant name "b" doesn't conform to UPPER_CASE naming style (invalid-name)

-----
Your code has been rated at 0.00/10
```

In the pylint 2.4.4 version, you will get a report as shown below. Messages might change depending on the version.

Score for the code given above is 0.0/10.0(Very low). If we get a low score, it doesn't mean that our code is wrong. The score represents how good/bad your code is understandable by another programmer. We need to improve our code by considering the suggestions given in the report and if the score is less, we use better commenting to improve the score.

The modified version of the code is:

```
'''
This program adds two numbers and displays their results
'''

A = 1
B = 2
print('Sum of Numbers:', A + B)
```

If we run the above code using pylint, we will get the following result

```
C:\Users\krubhuu_2000\Desktop>pylint add.py

-----
Your code has been rated at 10.00/10 (previous run: 6.67/10, +3.33)
```

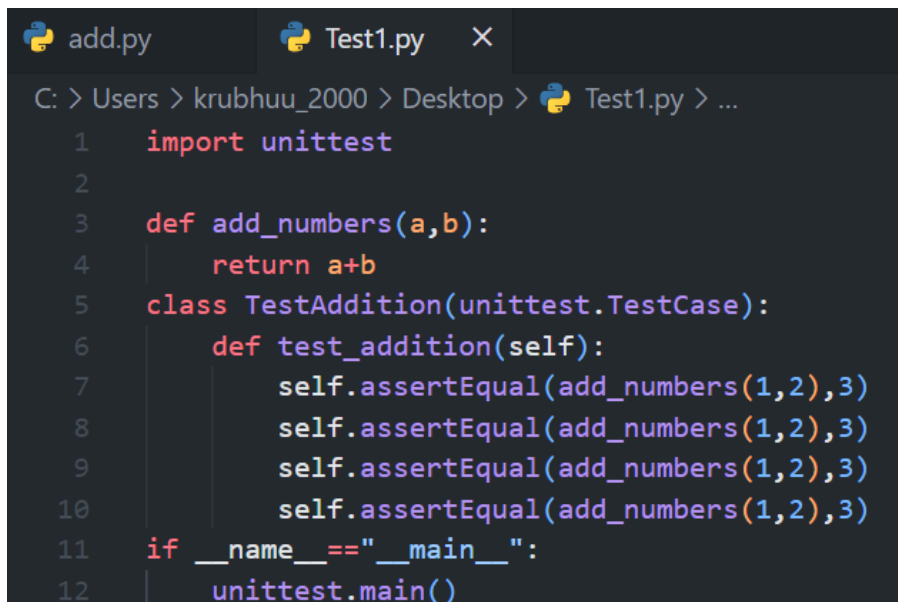
Here we improved our score from 0.0 to 10.0

2.PyUnit:

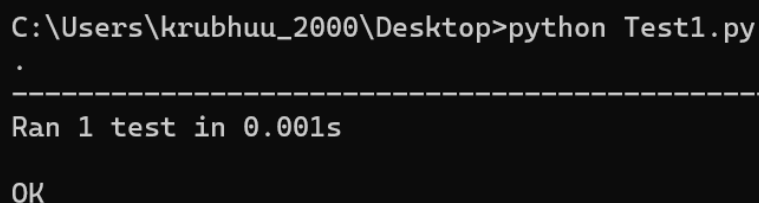
PyUnit is a tool used in software engineering for unit testing of the developed code where it is used for line-by-line testing of code and checking for the functionality of the code.

You can first write unit tests as code. Then, run that test code automatically every time you make changes in the software code. This way, if a test fails, you can quickly isolate the area of the code that has the bug or error.

Syntax for unit-test:



```
add.py Test1.py X
C: > Users > krubhuu_2000 > Desktop > Test1.py > ...
1  import unittest
2
3  def add_numbers(a,b):
4      return a+b
5  class TestAddition(unittest.TestCase):
6      def test_addition(self):
7          self.assertEqual(add_numbers(1,2),3)
8          self.assertEqual(add_numbers(1,2),3)
9          self.assertEqual(add_numbers(1,2),3)
10         self.assertEqual(add_numbers(1,2),3)
11  if __name__=="__main__":
12      unittest.main()
```



```
C:\Users\krubhuu_2000\Desktop>python Test1.py
.
-----
Ran 1 test in 0.001s
OK
```

3.Git/Git hub:

Git is a powerful tool that helps developers keep track of changes in their code. Imagine you're writing a book, and Git allows you to save different versions of your chapters, so you can always go back and see what you changed or fix mistakes. It's like having an unlimited undo button for your code.

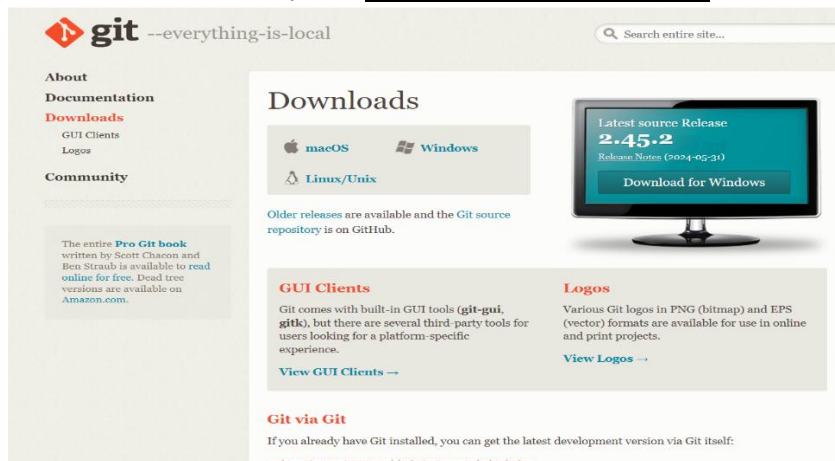
GitHub is also called git in cloud. It is a website where you can store your code using Git. It's like a library where you can share your projects with others, collaborate on code with teammates, and even contribute to other people's projects. GitHub makes it easy to work together and keep track of everyone's contributions.



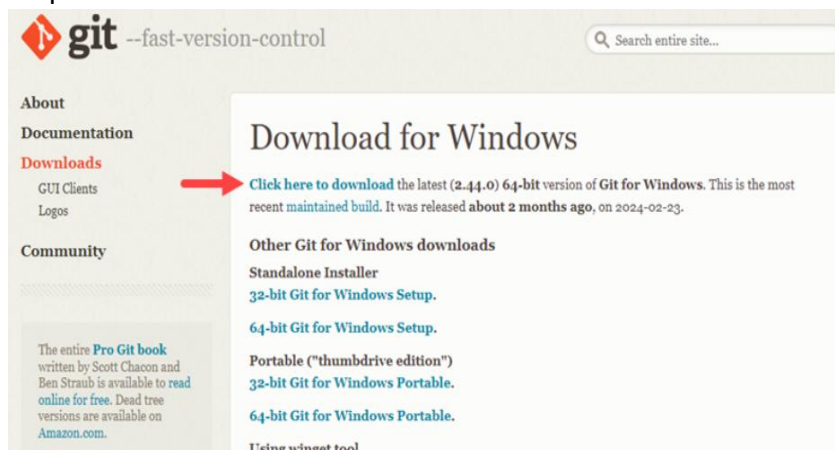
INSTALLATION:

How to install git on Windows's

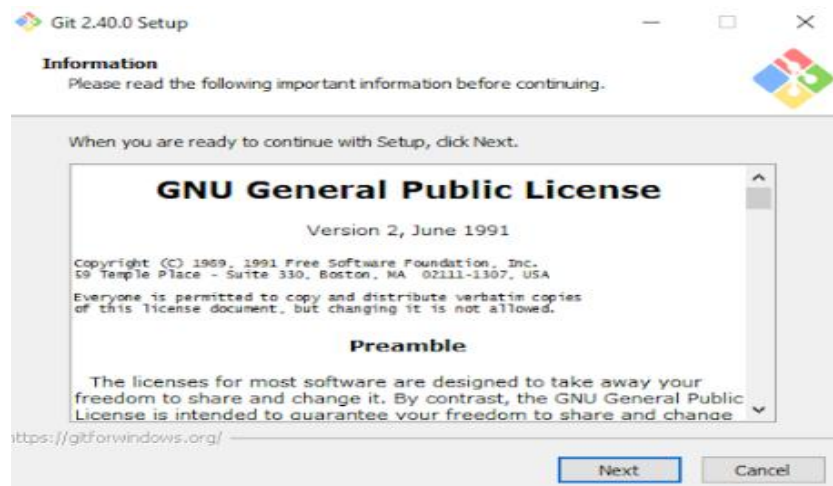
STEP 1: Install the Git, visit <https://www.git-scm.com/>



Step 2: click download for Windows.

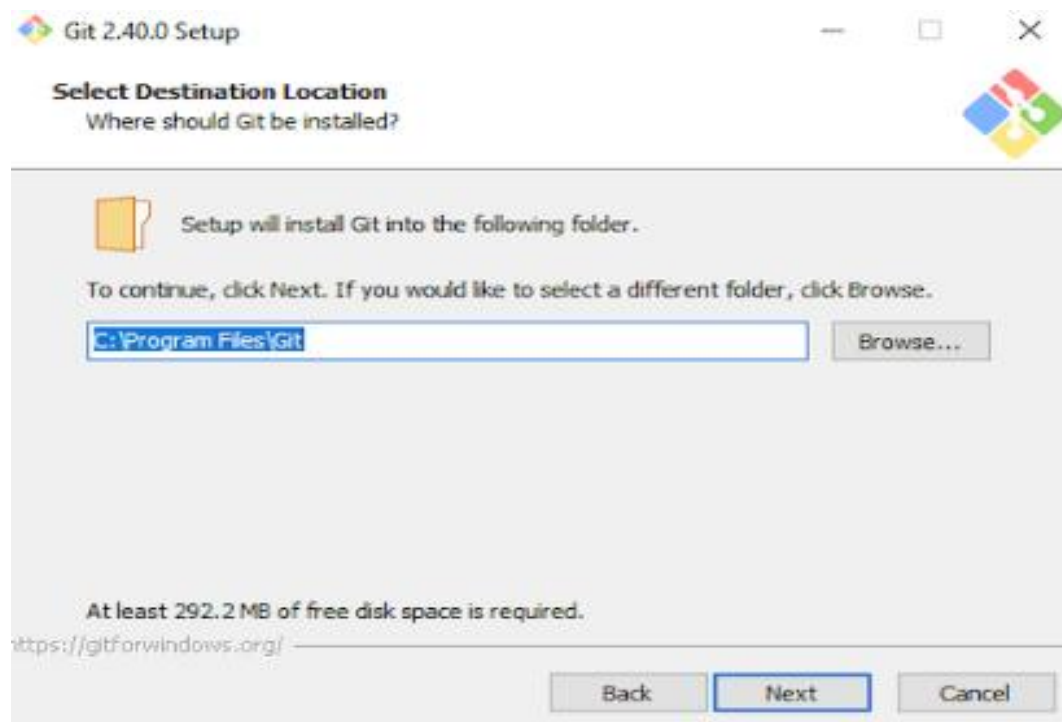


Step 3: Run the Installer and Select Option.

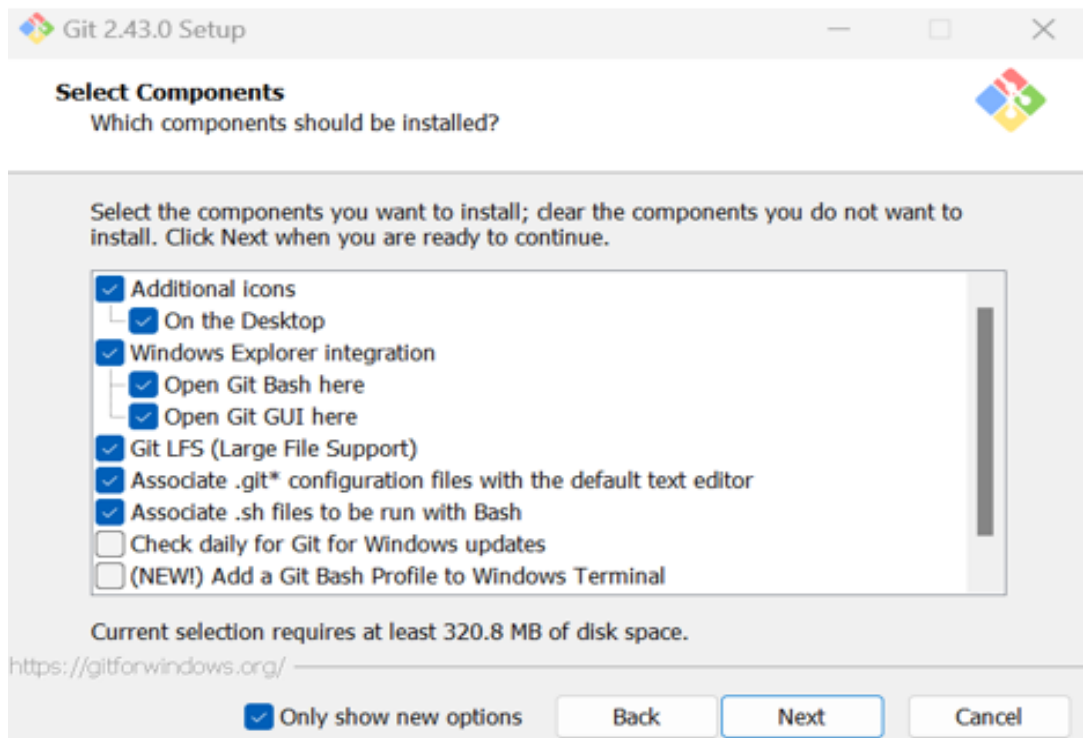


Read the GNU General Public License, and when you're prepared to install, click Next.

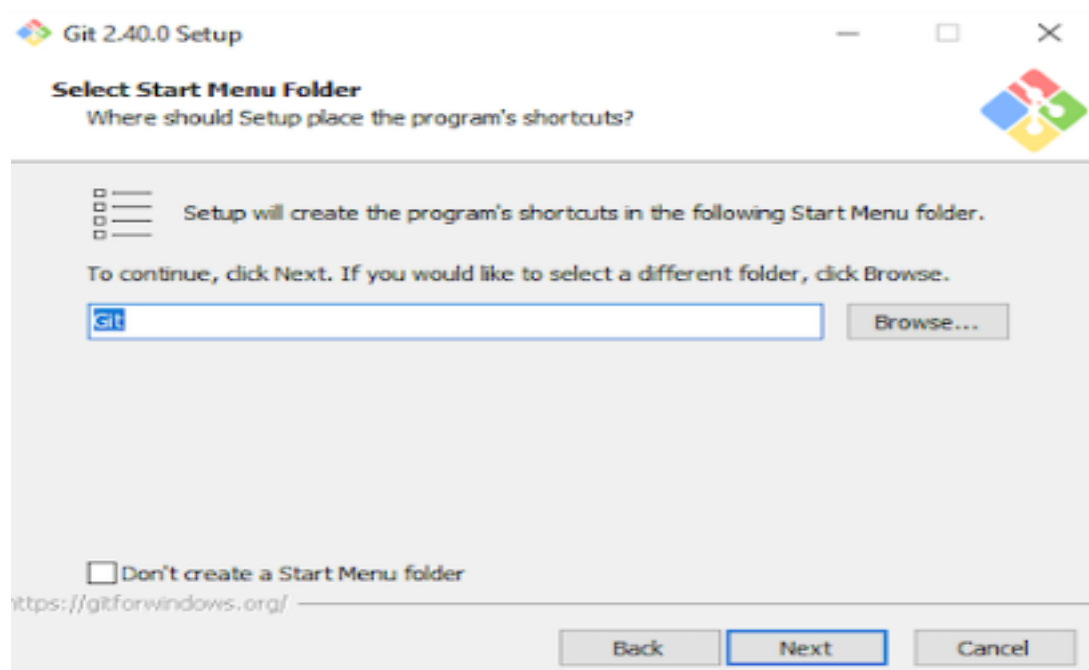
Step 4: The installer will request an installation location. Keep the default unless you need to change it, and click Next.



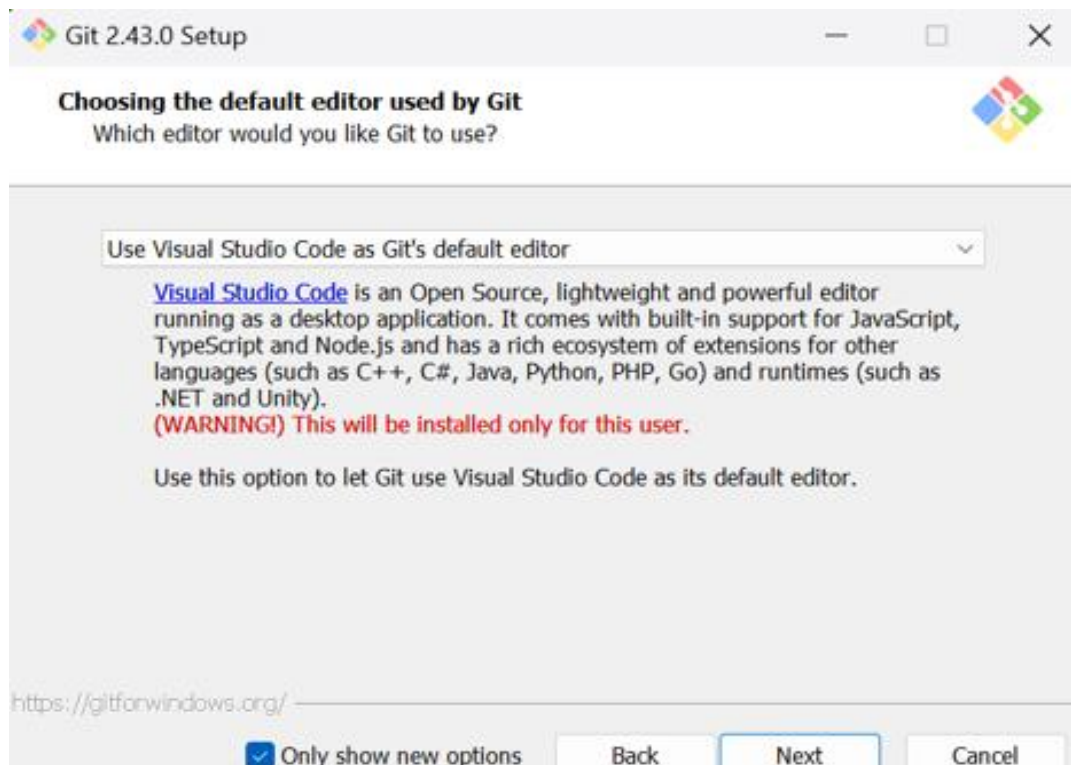
Step 5: A component selection screen will be displayed. Keep the default settings unless you need to modify them, and click Next.



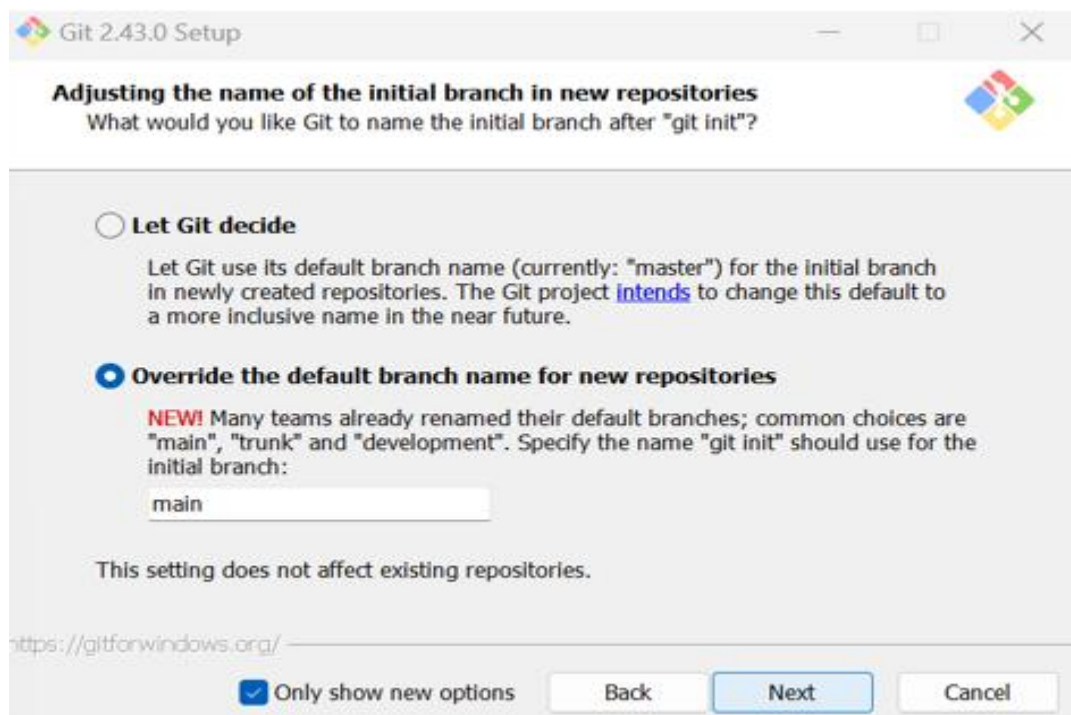
Step 6: You'll be prompted to create a start folder. Leave it as is and click Next



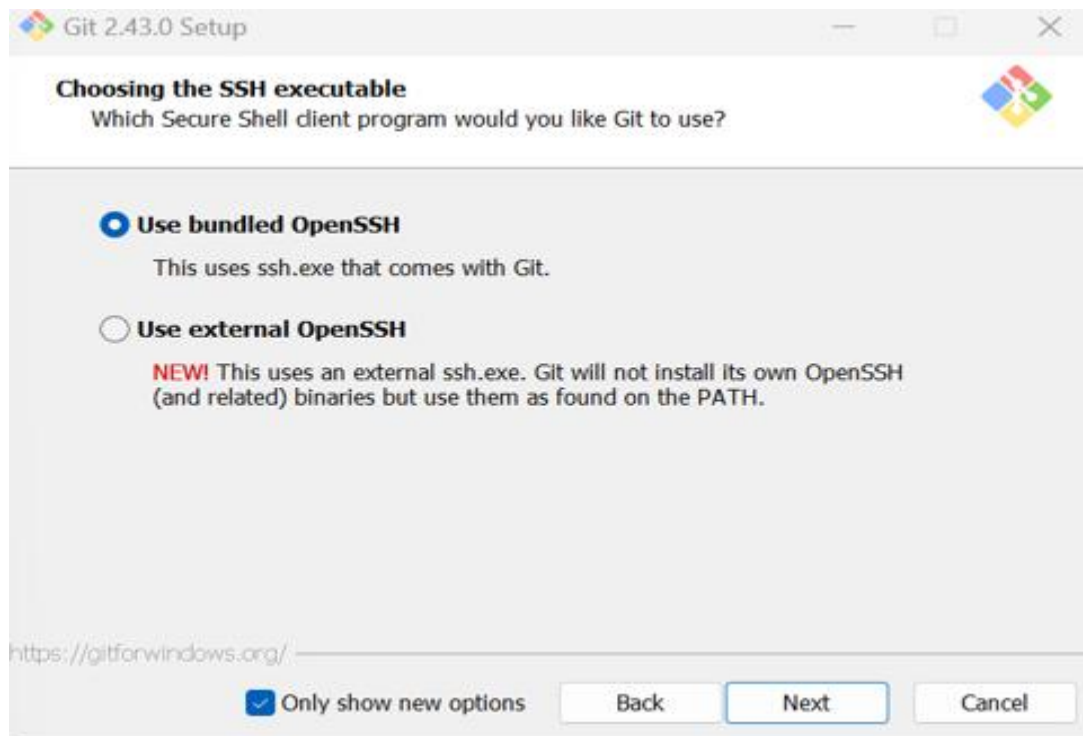
Step 7: Choose a text editor to use with Git. Click on the drop-down menu to pick the text editor you like to use like Vim, Notepad++, etc and click Next.



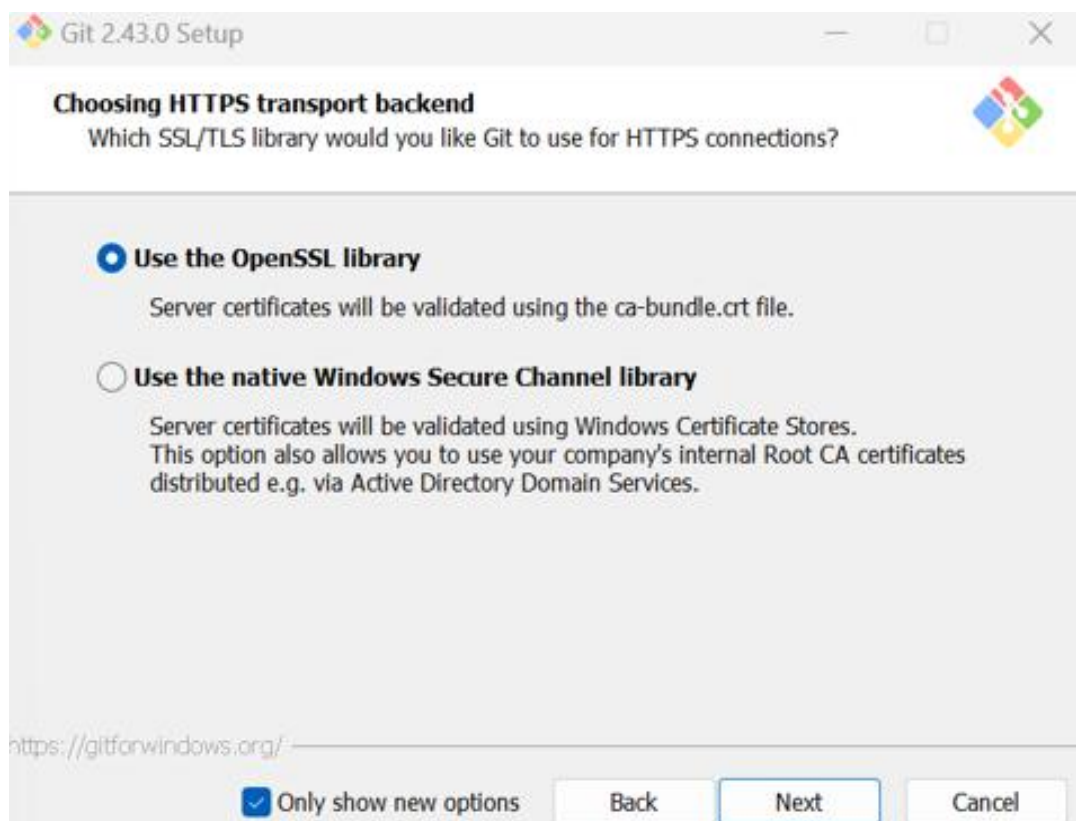
Step 8: Click next and set the branch name as main.



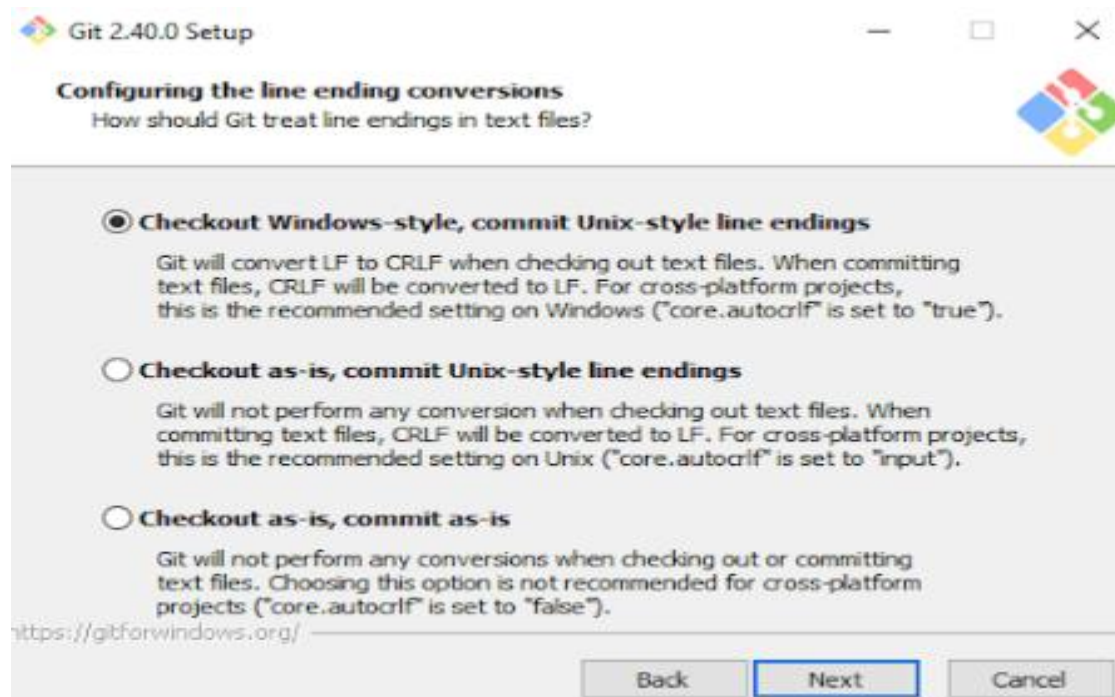
Step 9: select the default setting and click next.



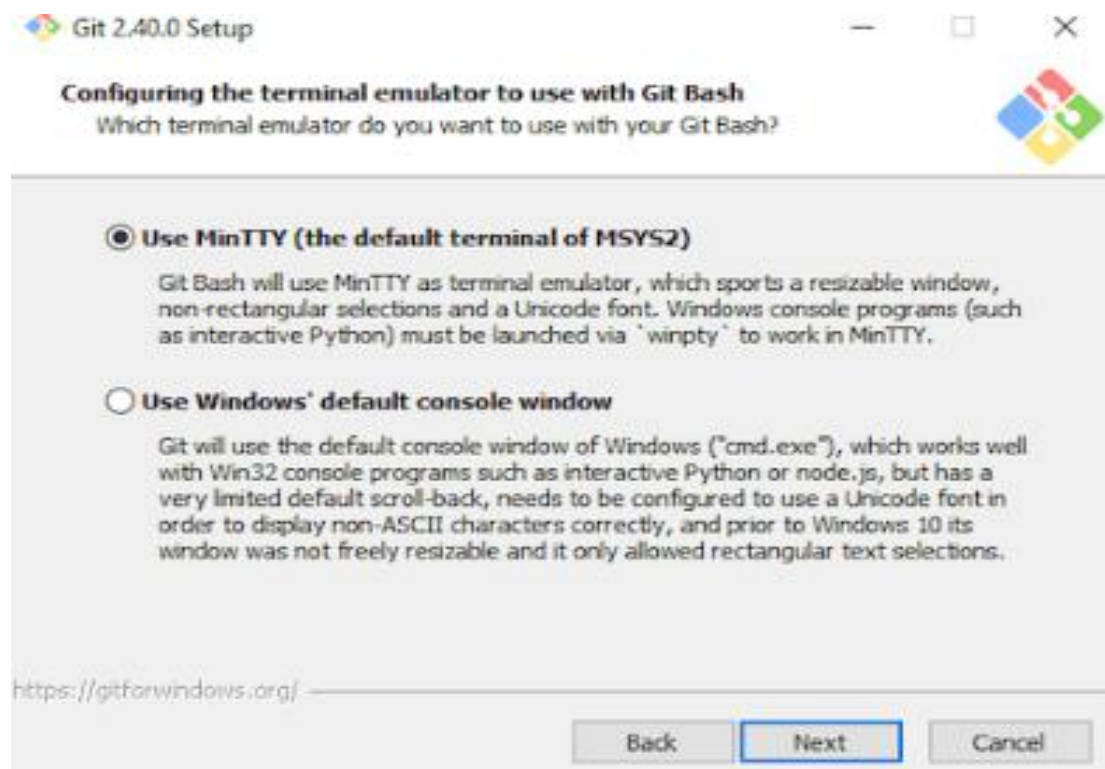
Step 10: select the default OpenSSL library and, then click next.



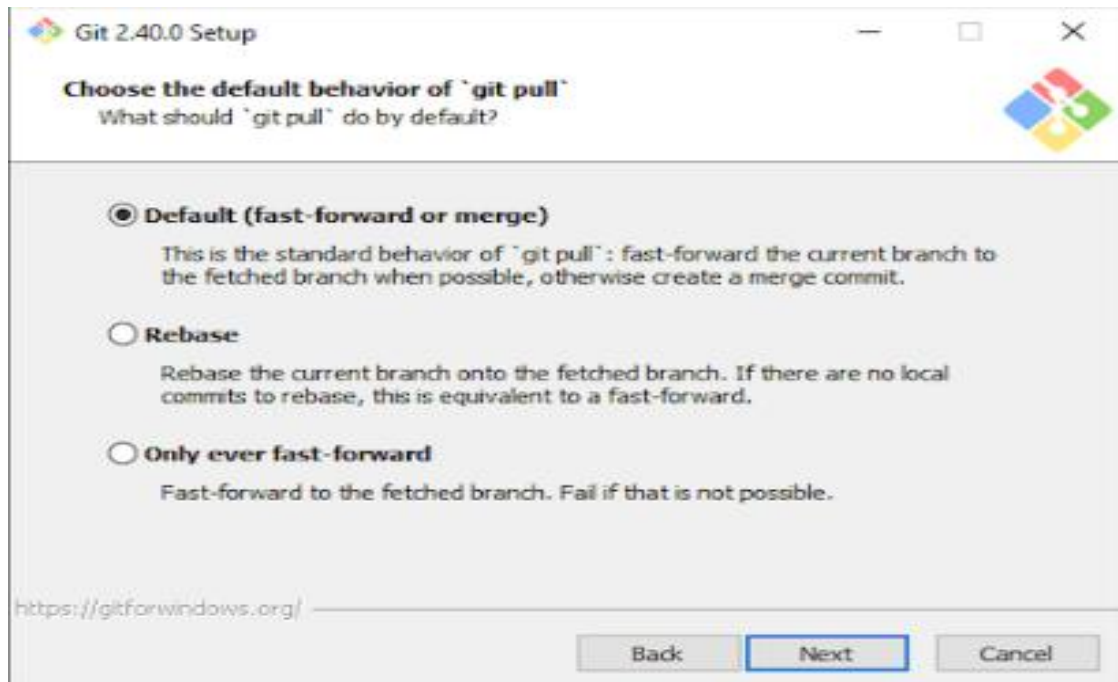
step 11: Checkout Windows-style, commit Unix-style line endings. Changing this option might cause issues. Click Next.



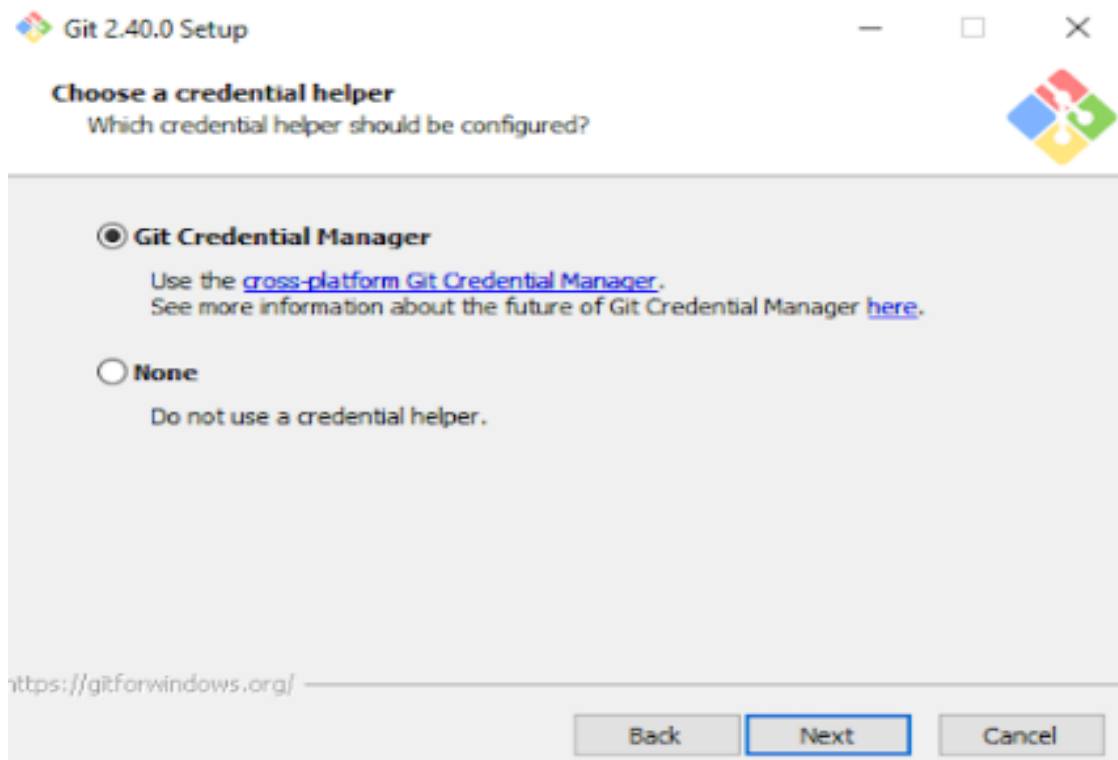
Step 12: select the terminal emulator. The default MinTTY is recommended. Click Next.



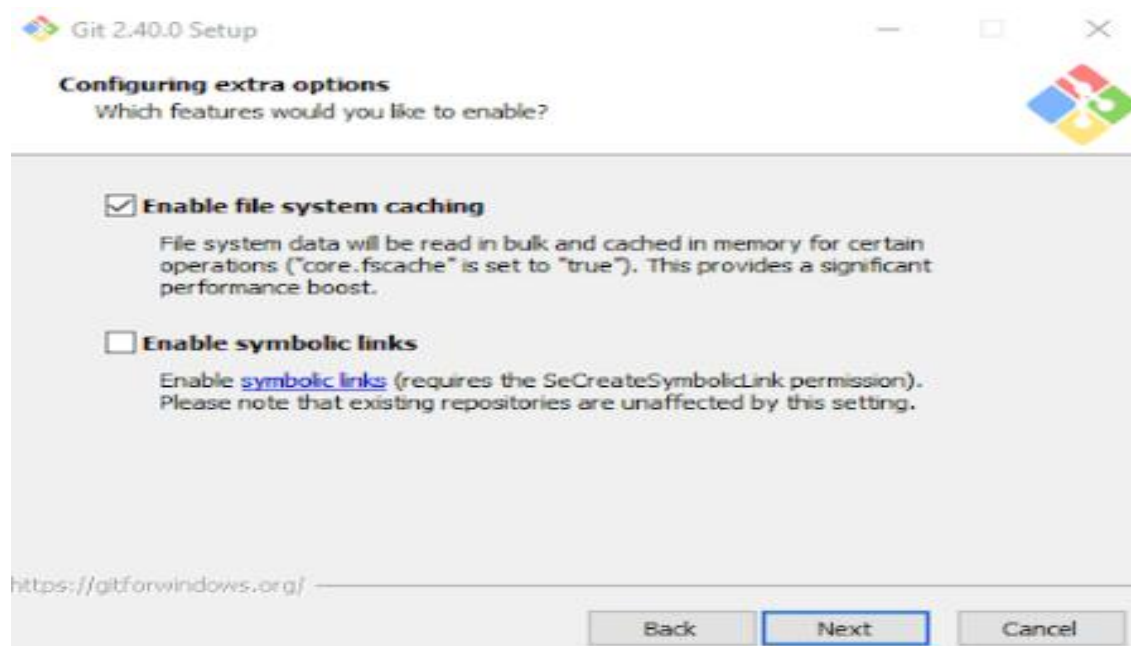
Step 13: ask what the git pull command should do. Again, the default option is recommended. Click Next.



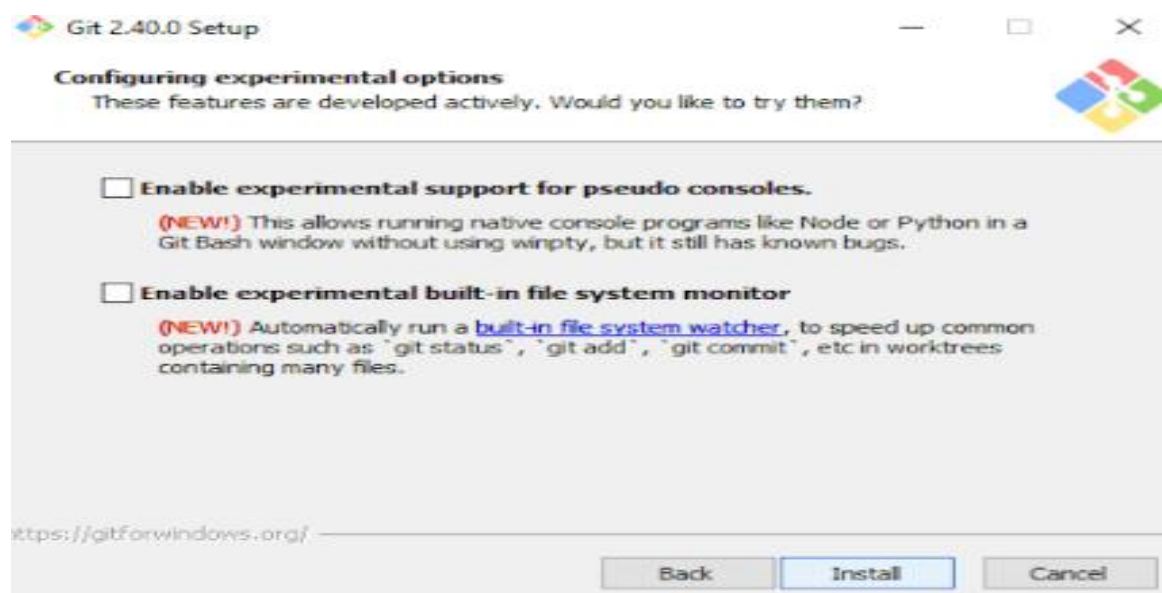
Step 14: You'll need to choose which credential helper to use next. Keep the default option selected and click Next.



Step 15: Next, it'll be presented with some extra options to customize your installation. Once done making your selections, click Next.



Step 16: Click on, Support for pseudo consoles or Built-in file system monitor. Check the boxes to whichever you'd like (or none), then click Install.



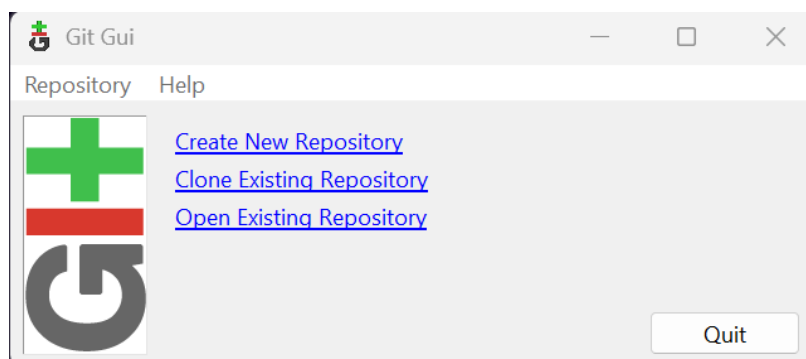
On the last screen of the installer, you can opt to view the Release Notes or Launch Git Bash.

Step 17: Verify the installation with Git Bash

To ensure that Git has been installed correctly, open Git Bash and type the following command: `git --version`
Press **Enter**, and the name of the version of Git should appear.

```
krubhuu_2000@Krubhuu MINGW64 ~  
$ git --version  
git version 2.47.1.windows.2
```

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git --version  
git version 2.47.1.windows.2
```



How to push a local python file from Git-to-Git hub:

Step 1: Initialise git by using `$ git init` command in command prompt or in git bash

```
C:\Users\krubhuu_2000>git init  
Initialized empty Git repository in C:/Users/krubhuu_2000/.git/
```

Step 2: list all the files in the current folder where the python file is to check for the file

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>dir  
Volume in drive C is Windows  
Volume Serial Number is 384C-8577  
  
Directory of C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS  
  
05-04-2025  20:45    <DIR>          .  
05-04-2025  20:41    <DIR>          ..  
05-04-2025  20:36                49 add.py  
05-04-2025  20:34               359 Test1.py  
                2 File(s)              408 bytes  
                2 Dir(s)  19,878,596,608 bytes free
```

Step 3: use \$ git add "file name.py" to add the file to git., here file name is **addition.py**.

Step 4: use git status to find the respected file is added or not (detecting changes).

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git add add.py

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   add.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Test1.py
```

Step 5: use \$ git commit -m "any name for calling the push" "file name.py" for committing your file

Step 6: specify user name and user email to the git using the below command.

Step 7: use the same git commit command and check the file is committed or not.

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git commit -m "adding initial file"
[master (root-commit) f729ffc] adding initial file
1 file changed, 4 insertions(+)
create mode 100644 add.py

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git config --global user.email "sukhithabhushan51@gmail.com"
```

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git config --global user.name "Sukhi"

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git add .

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git commit -m "adding test file"
[master 866bcla] adding test file
1 file changed, 12 insertions(+)
create mode 100644 Test1.py
```

Step 8: use this command to open the repository \$ git remote add origin [https://\[TOKEN ID\]@github.com/\[REPO OWNER\]/\[REPO-NAME\]](https://[TOKEN ID]@github.com/[REPO OWNER]/[REPO-NAME])

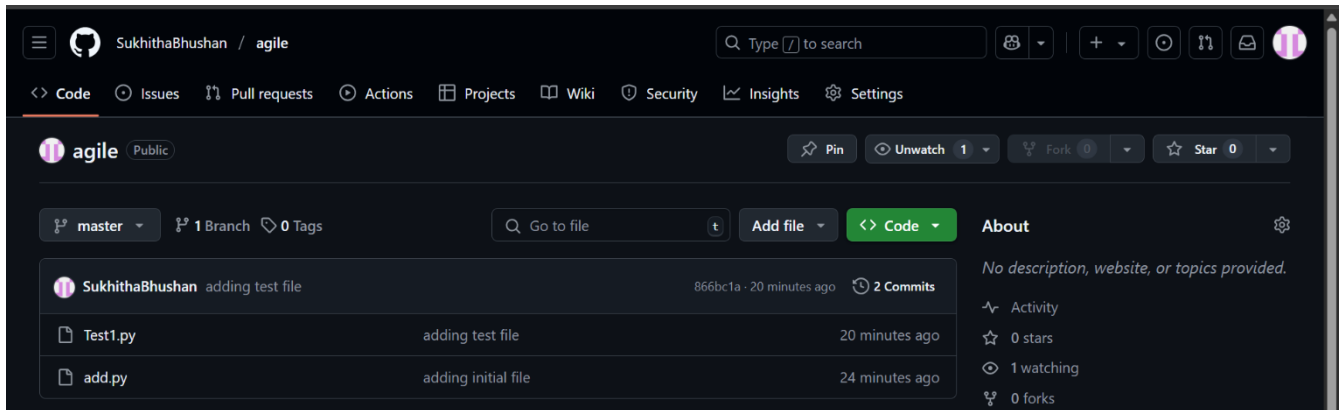
Step 9: use \$ git push -u origin master/main for pushing the file to the GitHub repository.

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git remote add origin https://github.com/SukhithaBhushan/agile.git

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git push -u origin master
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 649 bytes | 162.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/SukhithaBhushan/agile/pull/new/master
remote:
To https://github.com/SukhithaBhushan/agile.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git push origin master
Everything up-to-date
```

Step 10: now you can see the python file name **addition.py** in the GitHub repository being pushed from git to GitHub.



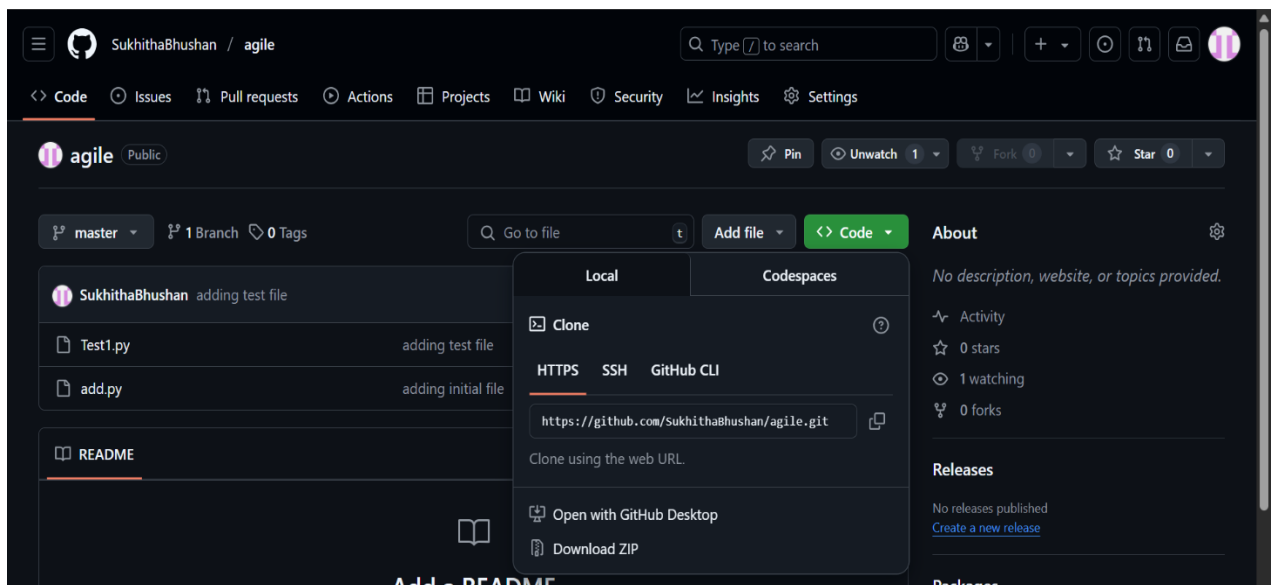
How to pull a file from GitHub to your local machine:

Step 1: Use cd to change to the directory where you want to store the GitHub repository

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>cd C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS\
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>.git
```

Step 2: **Clone the Repository:**

Use git clone to pull the entire repository from GitHub

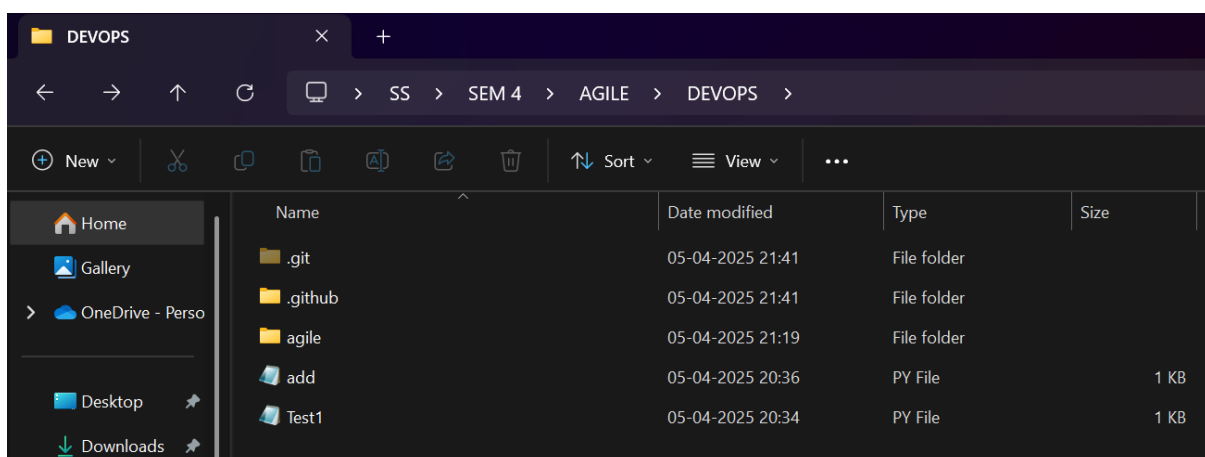
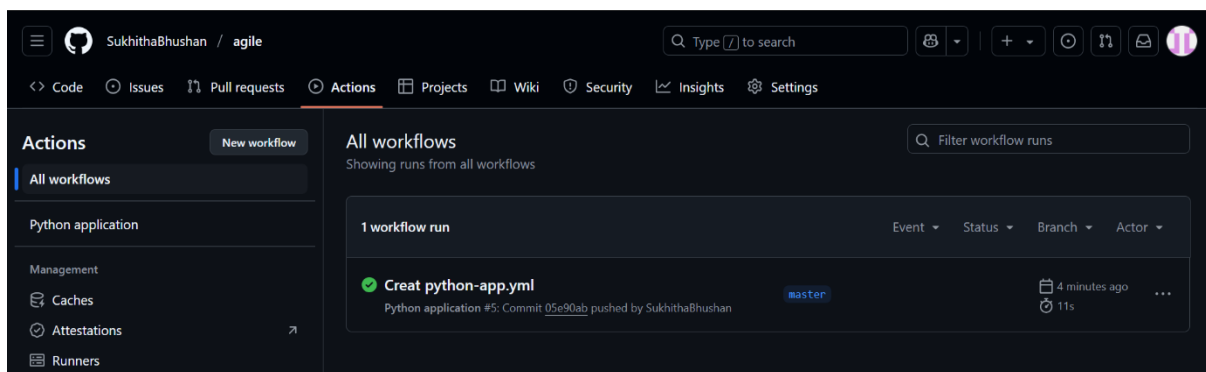


Step 3: git clone

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git clone https://github.com/SukhithaBhushan/agile.git
Cloning into 'agile'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

Step 4: To update your local copy with the latest changes from the remote repository, use:

```
C:\Users\krubhuu_2000\Desktop\SS\SEM 4\AGILE\DEVOPS>git pull
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 2.64 KiB | 37.00 KiB/s, done.
From https://github.com/SukhithaBhushan/agile
 866bc1a..05e90ab master    -> origin/master
Updating 866bc1a..05e90ab
Fast-forward
 .github/workflows/python-app.yml | 40 ++++++
 1 file changed, 40 insertions(+)
 create mode 100644 .github/workflows/python-app.yml
```



SulkhithaBhushan / agile

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

← Python application

✓

Creat python-app.yml #5

Summary

Jobs

✓

build

Run details

Usage

Workflow file

Workflow file for this run

github/workflows/python-app.yml at 05e90ab

```

1  # This workflow will install Python dependencies, run tests and lint with a single version of Python
2  # For more information see: https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python
3
4  name: Python application
5
6  on:
7    push:
8      branches: [ "master" ]
9    pull_request:
10     branches: [ "master" ]
11
12  permissions:
13    contents: read
14
15  jobs:
16    build:
17
18     runs-on: ubuntu-latest
19
20     steps:
21     - uses: actions/checkout@v4
22     - name: Set up Python 3.10
23       uses: actions/setup-python@v3
24       with:
25         python-version: "3.10"
26     - name: Install dependencies
27       run: |
28         python -m pip install --upgrade pip
29         pip install flake8 pytest
30         if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
31     - name: Lint with flake8
32       run: |
33         # stop the build if there are Python syntax errors or undefined names
34         flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
35         # exit-zero treats all errors as warnings. The GitHub editor is 127 chars wide
36         flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --statistics
37     - name: Test with unittest
38       run: |
39         python -m unittest discover
40

```

4.DOCKER/DOCKER HUB:

How to Install Docker in Window



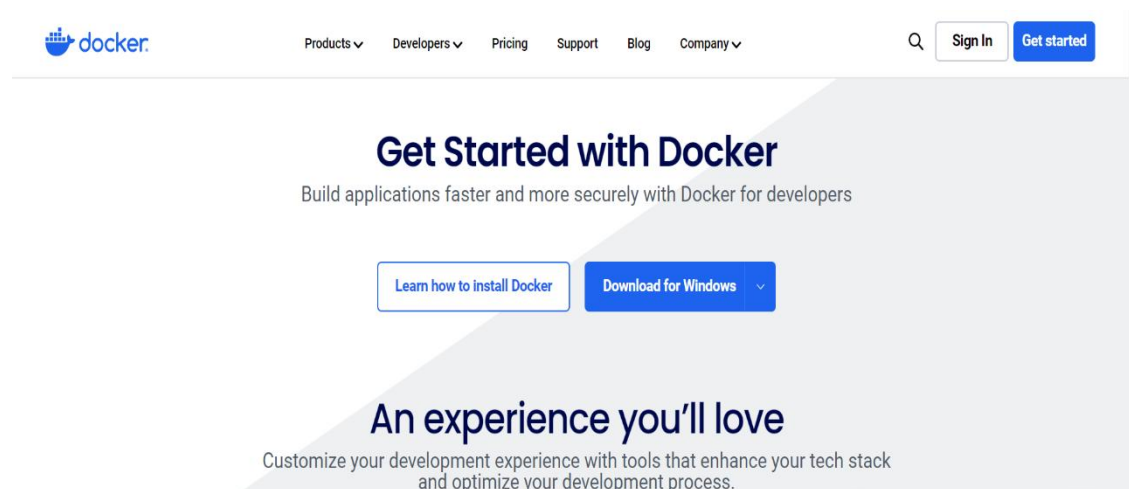
Step by Step guide on How to install Docker on Windows 10/11

Requirements:

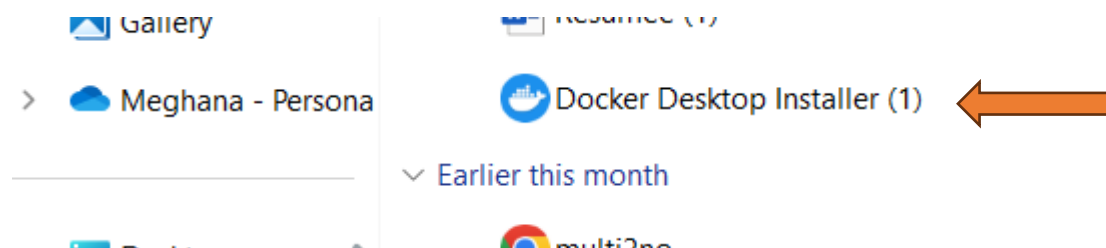
1. Windows 10 with 64 bits
2. Windows 11 with 64-bit operating system, it must be having higher level of translation.

Step 1: Downloading Docker

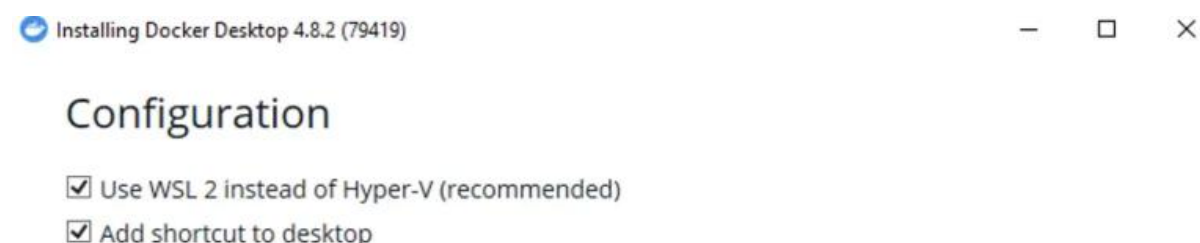
The first place to start is the [official Docker website](https://www.docker.com) from where we can download Docker Desktop.



open the downloaded installer and begin the installation.

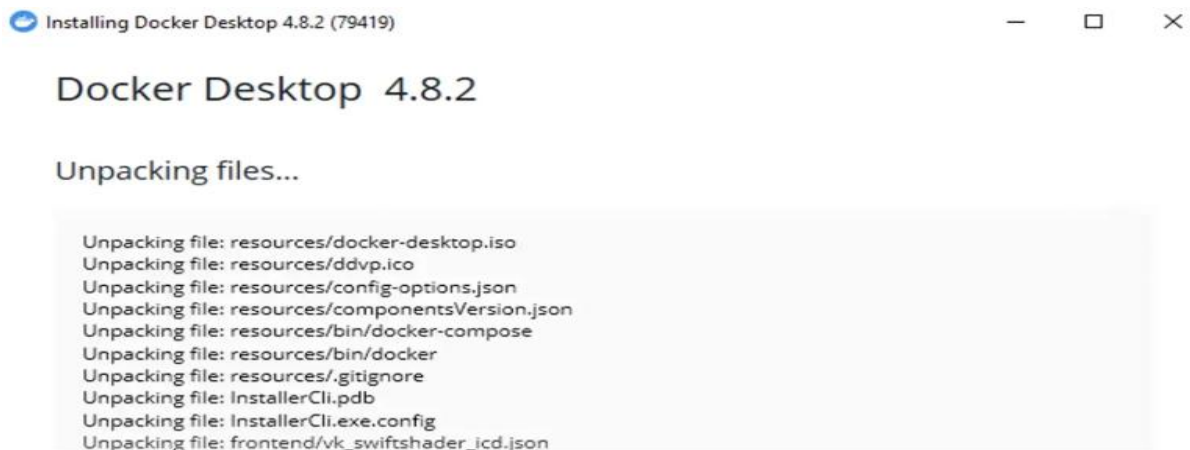


Step 2: Configuration To run Linux on Windows, Docker requires a virtualization engine. Docker recommends using WSL 2 as the preferred virtualization engine.

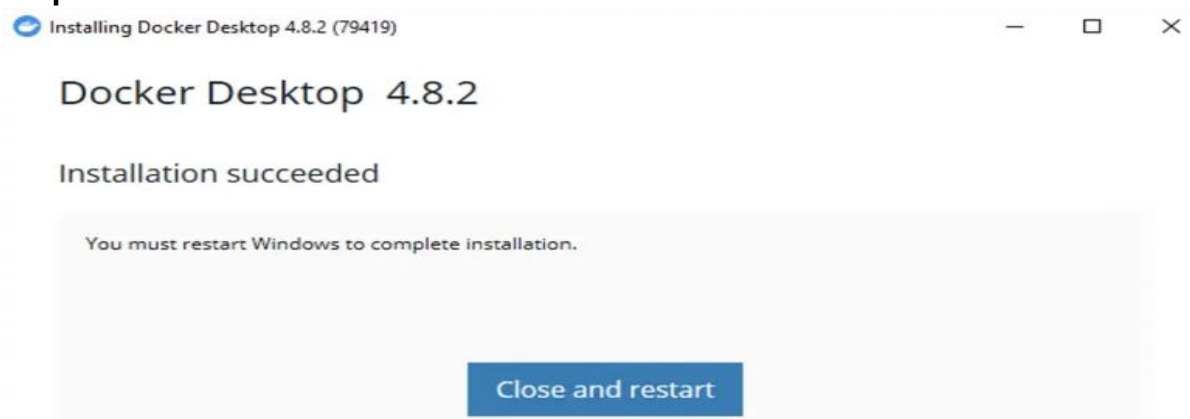


Step 3: Running the installation

Click Ok, and wait a bit...

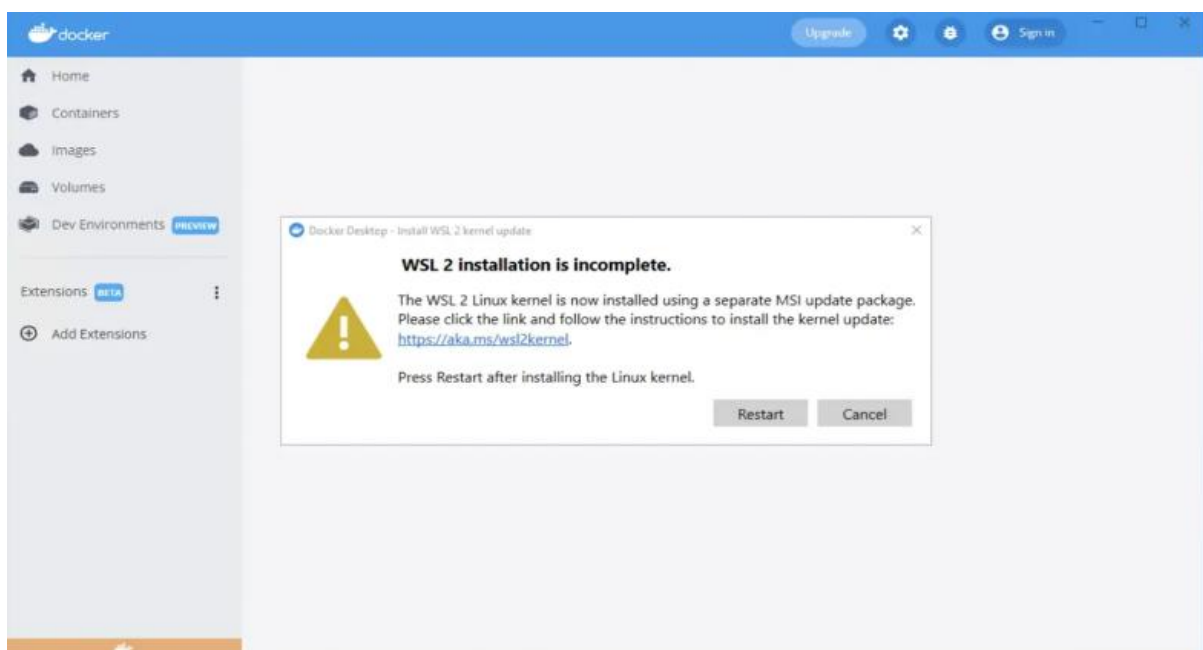


Step 4: Restart



Step 5: WSL 2 installation

After you accept the license terms, the Docker Desktop window will open. Since we have selected WSL 2 as our virtualization engine. **Don't click Restart just yet!**



Follow the link in the dialog window and download WSL 2.

Step 4 - Download the Linux kernel update package

1. Download the latest package:

- [WSL2 Linux kernel update package for x64 machines](#) 

Note

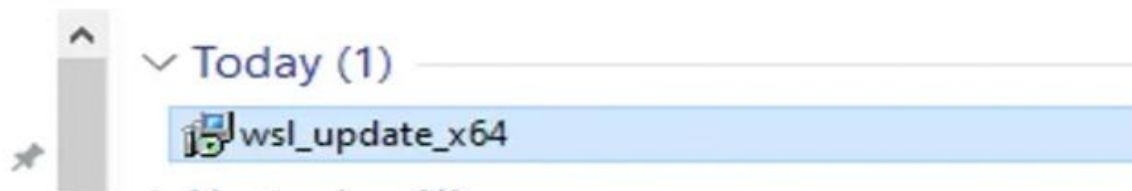
If you're using an ARM64 machine, please download the [ARM64 package](#) instead. If you're not sure what kind of machine you have, open Command Prompt or PowerShell and enter: `systeminfo | find "System Type"`. **Caveat:** On non-English Windows versions, you might have to modify the search text, translating the "System Type" string. You may also need to escape the quotations for the find command. For example, in German `systeminfo | find '"Systemtyp"'`.

2. Run the update package downloaded in the previous step. (Double-click to run - you will be prompted for elevated permissions, select 'yes' to approve this installation.)

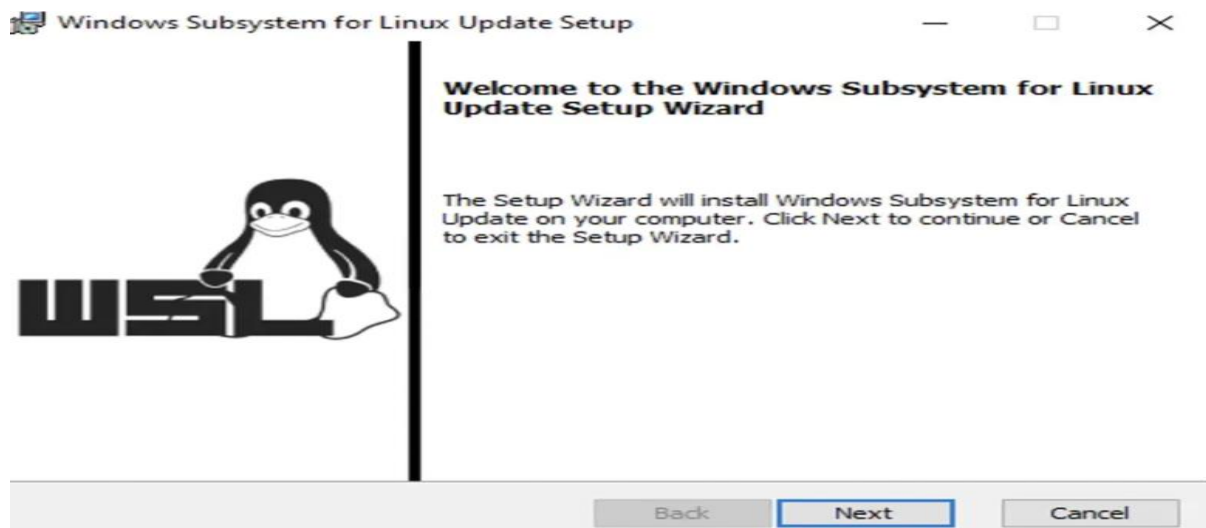
Once the installation is complete, move on to the next step - setting WSL 2 as your default version when installing new Linux distributions. (Skip this step if you want your new Linux installs to be set to WSL 1).

Open the installer.

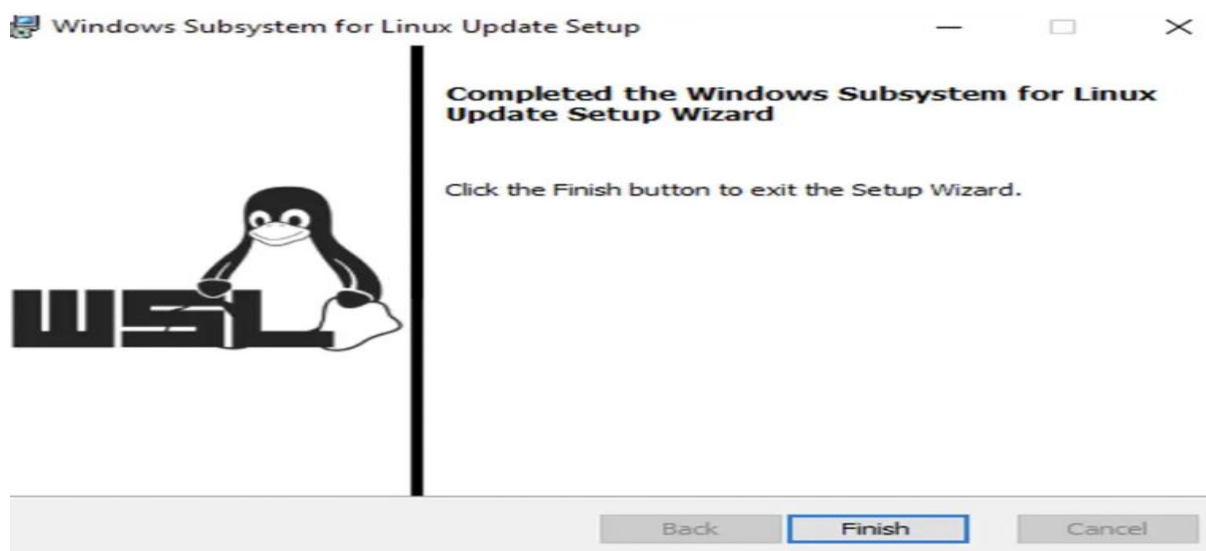
> Downloads



Click on *Next* to begin installing the Windows Subsystem for Linux (WSL).



After a few seconds, the installation should complete. So you may click on Finish.



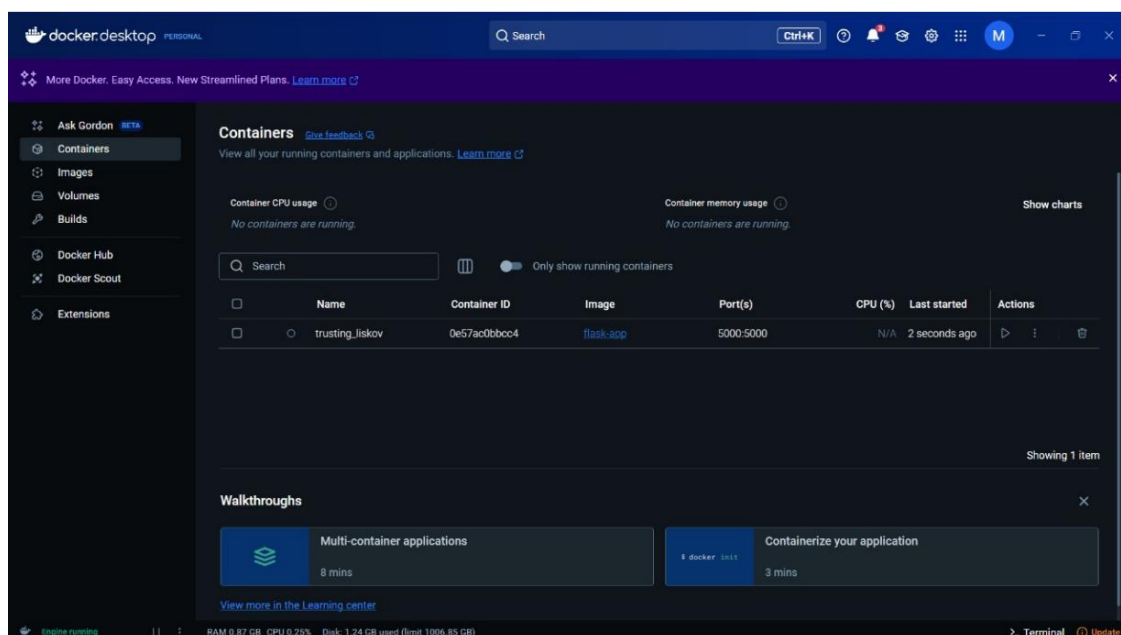
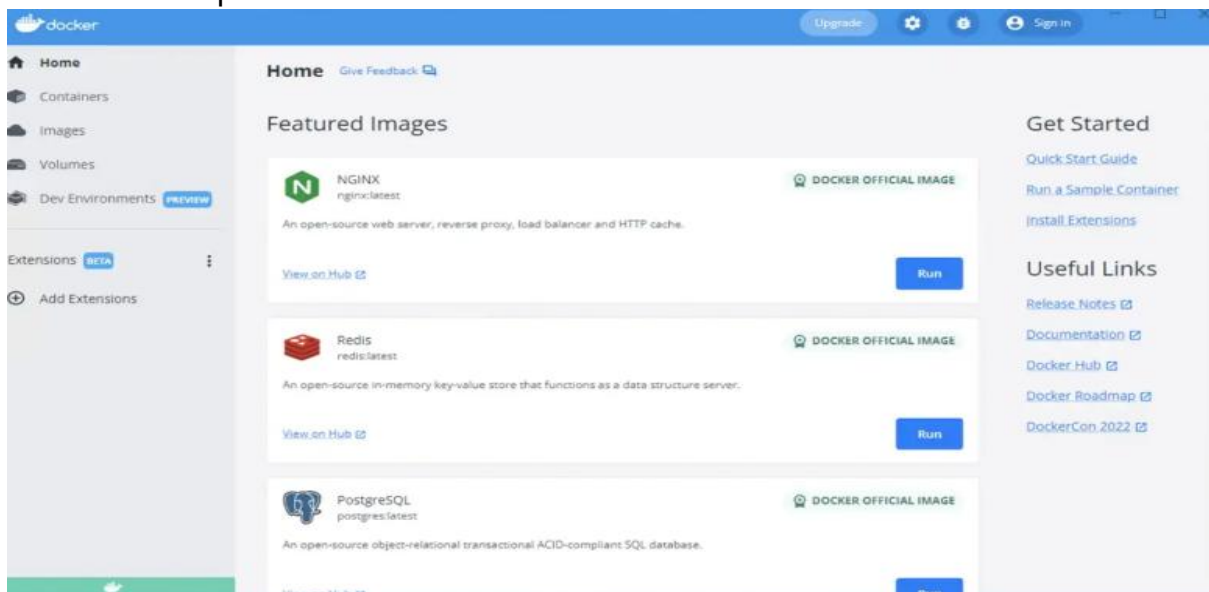
If you still have the Docker Desktop dialog window in the background, click on Restart.



Step 7 — Starting Docker Desktop



Docker Desktop screen should look like this.



Step-by-Step Guide to Create, Build and Run a Docker Flask Application

Step 1: Set Up the Flask Application

- ❑ Create a new directory for your project.
- ❑ In this directory, create a file named app.py with the following content:

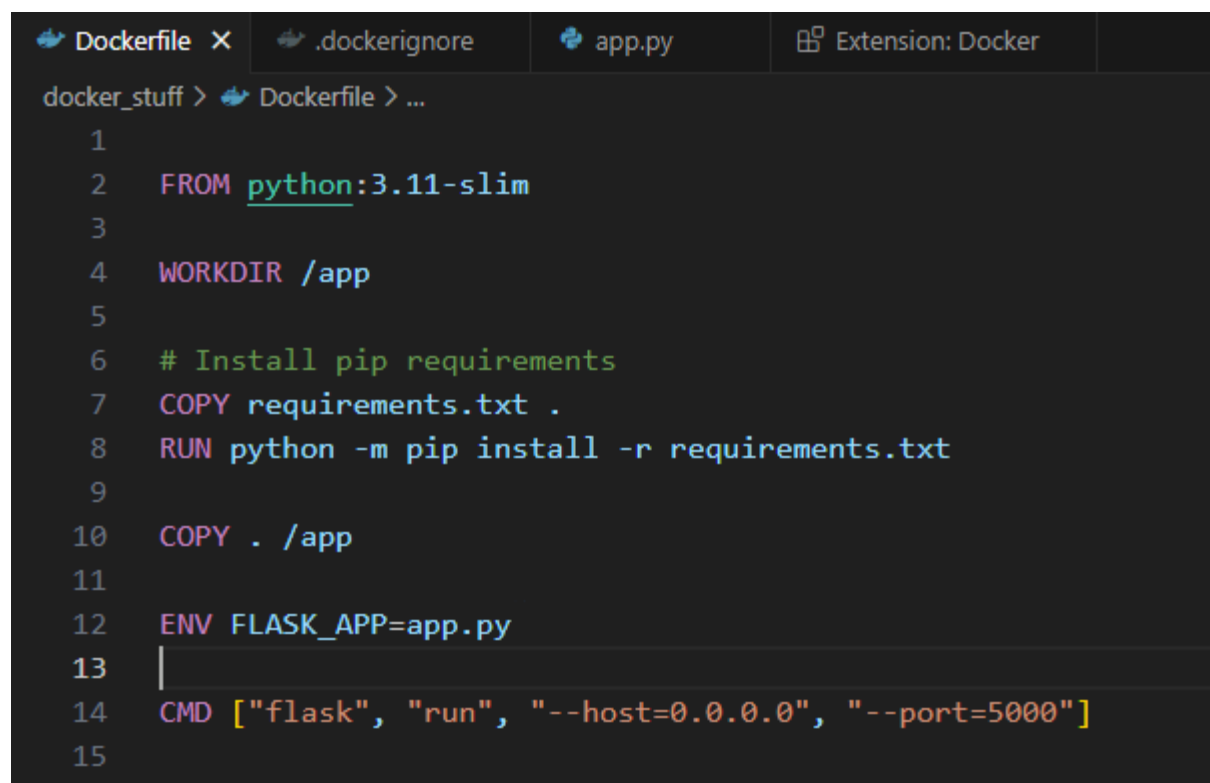
```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'
```

Step 2: Create the Docker file

- In the same directory, create a file named Docker file with no extension.
- Add the following instructions to the Docker file:

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'Dockerfile' (active), '.dockerignore', 'app.py', and 'Extension: Docker'. The 'Dockerfile' tab shows the following content:

```
docker_stuff > Dockerfile > ...
1
2 FROM python:3.11-slim
3
4 WORKDIR /app
5
6 # Install pip requirements
7 COPY requirements.txt .
8 RUN python -m pip install -r requirements.txt
9
10 COPY . /app
11
12 ENV FLASK_APP=app.py
13
14 CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
15
```

Step 3: Create .docker ignore File

- In the same directory, create a file named .dockerignore.
- Add the following content to .dockerignore to exclude the Docker file

```
Dockerfile .dockerignore X
docker_stuff > .dockerignore
1 Dockerfile
2 **/__pycache__
3 **/.venv
4 **/.classpath
5 **/.dockerignore
6 **/.env
7 **/.git
8 **/.gitignore
9 **/.project
10 **/.settings
11 **/.toolstarget
12 **/.vs
13 **/.vscode
14 **/*.proj.user
15 **/*.dbmdl
16 **/*.jfm
17 **/bin
18 **/charts
19 **/docker-compose*
20 **/compose*
21 **/Dockerfile*
22 **/node_modules
23 **/npm-debug.log
24 **/obj
25 **/secrets.dev.yaml
26 **/values.dev.yaml
27 LICENSE
28 README.md
```

Requirements.txt:

```
Dockerfile .dockerignore requirements.txt X
docker_stuff > requirements.txt
1 flask==3.0.1
2
```

Step 4: List Directory Contents

Mode	LastWriteTime	Length	Name
-a----	4/3/2025 11:07 PM	379	.dockerignore
-a----	4/3/2025 10:58 PM	108	app.py
-a----	4/3/2025 11:07 PM	375	Dockerfile
-a----	4/3/2025 10:59 PM	212	requirements.txt

This command lists the contents of the current directory.

Step 5: Display Contents of app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'
```


This command shows the contents of the app.py file.

Step 6: Display Contents of Docker file

```
FROM python:3.11-slim

WORKDIR /app

# Install pip requirements
COPY requirements.txt .
RUN python -m pip install -r requirements.txt

COPY . /app

ENV FLASK_APP=app.py

CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
```

This command shows the contents of the Docker file.

Step 7: Build the Docker Image

```
PS C:\Users\aryan\OneDrive\Desktop\COLLEGE\sem4\AGILE\docker> docker build -t flask-app .
[+] Building 32.6s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 408B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 3.8s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.0s
=> [1/4] FROM docker.io/library/python:3.11-slim@sha256:7029b00486ac40bed03e36775b864d3f3d39dcdbf19cd45e6a52d54 21.4s
=> => resolve docker.io/library/python:3.11-slim@sha256:7029b00486ac40bed03e36775b864d3f3d39dcdbf19cd45e6a52d541 0.1s
=> => sha256:b4a6013a75d147904664f2f911837e64b9e25586c4085e3fdb176719c7f5687 16.20MB / 16.20MB 9.1s
=> => sha256:7029b00486ac40bed03e36775b864d3f3d39dcdbf19cd45e6a52d541e6c178f0 9.13kB / 9.13kB 0.0s
=> => sha256:9a0a27e638a9b2da7384fc08fb72abab453ddadd2fe92aab6ed4a1ab67455e5f 1.75kB / 1.75kB 0.0s
=> => sha256:de3a6c124050bb59b294dd2c95caf61f7da984337aafad3c0a9a06854fa86880 5.29kB / 5.29kB 0.0s
=> => sha256:6e909acdb790c5a1989d9cfc795fda5a246ad6664bb27b5c688e2b734b2c5fad 28.20MB / 28.20MB 18.1s
=> => sha256:245f048c9f8a2432538751286248ab017d49f67a6bc8b70db1a05a20c562ec6f 3.51MB / 3.51MB 3.6s
=> => sha256:9eab895eb84c23106b53e9b3eaa76d790aa11ad723bc8d8e429785e790e5630d 249B / 249B 4.1s
=> => extracting sha256:6e909acdb790c5a1989d9cfc795fda5a246ad6664bb27b5c688e2b734b2c5fad 1.5s
=> => extracting sha256:245f048c9f8a2432538751286248ab017d49f67a6bc8b70db1a05a20c562ec6f 0.2s
=> => extracting sha256:b4a6013a75d147904664f2f911837e64b9e25586c4085e3fdb176719c7f5687 0.9s
=> => extracting sha256:9eab895eb84c23106b53e9b3eaa76d790aa11ad723bc8d8e429785e790e5630d 0.0s
=> [internal] load build context                                   0.1s
=> => transferring context: 618B                                     0.0s
=> [2/4] WORKDIR /app                                             0.2s
=> [3/4] RUN pip install flask                                    6.4s
=> [4/4] COPY . /app                                              0.1s
=> exporting to image                                             0.3s
=> => exporting layers                                              0.2s
=> => writing image sha256:4bdcffaba5ce11c935a52208df9785504a3565371478412e7626aa0833c988c6 0.0s
=> => naming to docker.io/library/flask-app                        0.0s
```

This command builds a Docker image from the Dockerfile in the current directory and tags it as sample-flask-app:v1

Step 8: List Docker Images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
sample-flask-app	v1	082c6b2d9cb2	14 seconds ago	145MB
<none>	<none>	d753fba50107	11 hours ago	145MB

This command lists all the Docker images on the system.

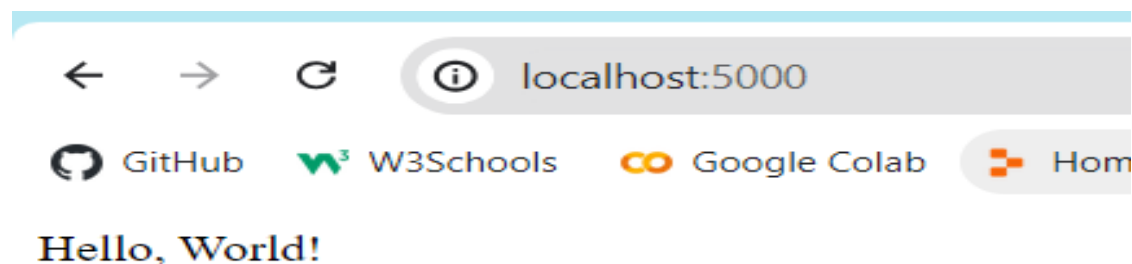
Step 9: Run the Docker Container

```
C:\Users\megha\directory>docker container run -d -p 5000:5000 sample-flask-app:v1  
acb8a092b33379a96096500c8fd17e116cce68b4ded2e3afa3beddc70d307594
```

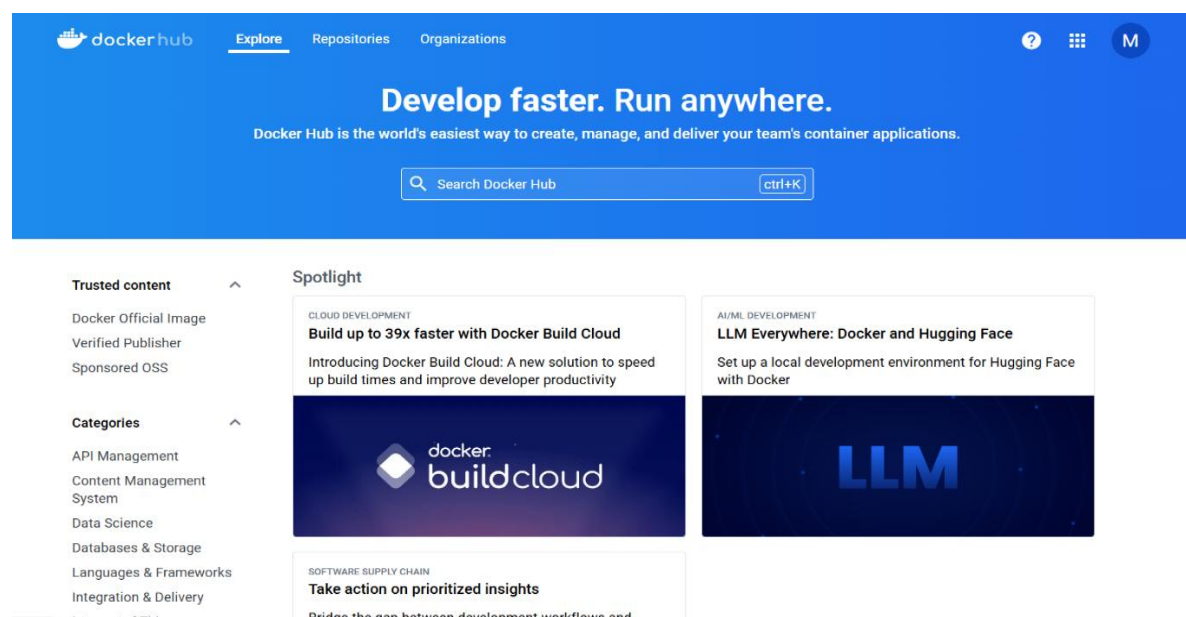
This command runs the sample-flask-app:v1 Docker image as a container, mapping port 5000 of the host to port 5000 of the container.

Step 10: List Running Docker Containers

Our Flask application is now running inside the container. To test it, open a web browser and go to <http://localhost:5000>. You should see the message **Hello, World!** displayed like this:



How to push docker image to Docker hub (Create repository in docker hub)



Create repository

Namespace: meghanas2004

Repository Name *

Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

☒ Public Appears in Docker Hub search results

☐ Private Only visible to you

[Cancel](#) [Create](#)

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

Step-by-Step Commands to Push Docker Image to Docker Hub

Step 1: Log in to Docker Hub

```
Authenticating with existing credentials... [Username: minarchie]
Info → To login with a different account, run 'docker logout' followed by 'docker login'
Login Succeeded
```

This command logs you into Docker Hub

Step 2: Tag the Docker Image

```
> docker tag sample-flask-app:v1 minarchie/sample-flask-app:v1
> []
```

This command tags the local Docker image `sample-flask-app:v1` with a new name `meghanas2004/sample-flask-app:v1`. The format is `docker tag [SOURCE_IMAGE] [TARGET_IMAGE]`

Step 3: Push the Docker Image to Docker Hub

This command pushes the tagged image `meghanas2004/sample-flask-app:v1` to your Docker Hub repository.

```

PS C:\Users\swath\RVU\academic\sem4\agile\devops_report\docker_stuff> docker push minarchie/sample-flask-app:v1
The push refers to repository [docker.io/minarchie/sample-flask-app]
6e909acdb790: Pushed
9eab895eb84c: Pushed
dfec7c2d03d4: Pushed
245f048c9f8a: Pushed
b4a6013a75d1: Pushed
d8b548280143: Pushed
3a120ac70266: Pushed
46a12c74ca4c: Pushed
6ad9786a0cdf: Pushed
v1: digest: sha256:deaa75e8380656e7ace326389d5fa2920c3f4b245f494e437ae3fea4fb22e024 size: 856

```

The screenshot shows the Docker Hub interface for the repository `minarchie/sample-flask-app`. The page includes a sidebar with navigation links, a main content area with repository details, and a 'Docker commands' section. The repository page shows the tag `v1` and a 'Docker commands' section with the command `docker push minarchie/sample-flask-app:tagname`.

Pull a Docker image from Docker Hub to your local machine:

Step1: Log in to Docker Hub

```

Authenticating with existing credentials... [Username: minarchie]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

```

Step 2: Pull the Docker Image:

- Use the `docker pull` command to download the image from Docker Hub

```

v1: Pulling from minarchie/sample-flask-app
Digest: sha256:deaa75e8380656e7ace326389d5fa2920c3f4b245f494e437ae3fea4fb22e024
Status: Image is up to date for minarchie/sample-flask-app:v1
docker.io/minarchie/sample-flask-app:v1

```

5.Kubernetes:

It is an open-source orchestration system for docker container, It is also called as container management tool.

Kubernetes was implemented by google but later donated to CNCF (Cloud Native Computer Foundation)

It eliminates the manual process involved in the deployment of the containerised applications.

Major functions of Kubernetes:

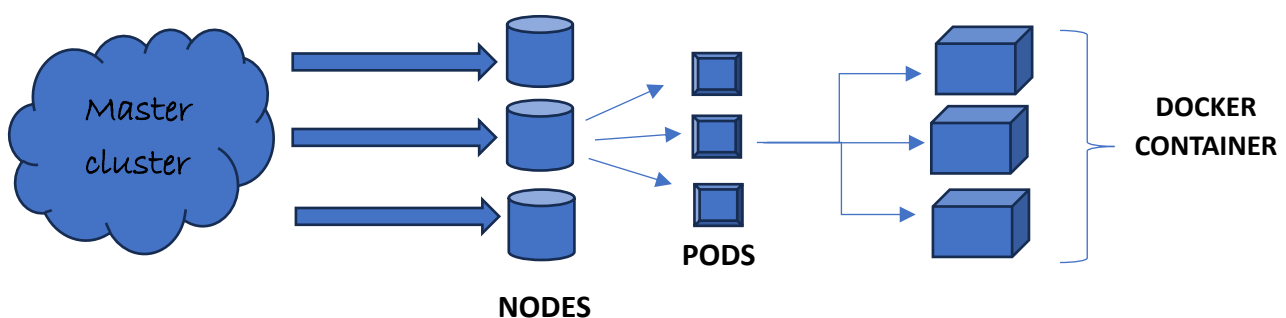
1. Deploying
2. Scheduling
3. Scaling
4. Load Balancing
5. Batch execution

Kubernetes has an option called Roll out and roll back mechanism which enables Zero down time deployment, which means our website or application is never down or in an unstable state during the process and also enables introducing new features easily or deliver safely.

Other alternates of Kubernetes:

Docker swam and Apache Mesos marathon

Kubernetes Architecture:



- **Nodes:** Physical or virtual machines that run applications and Kubernetes components.
- **Pods:** Smallest deployable units in Kubernetes, containing one or more containers.
- **Master-Cluster:** Collection of nodes that Kubernetes manages as a single unit.
- **Docker Container:** it contains developed application, virtual machine, operating system and DevOps required for the application all put in one container.

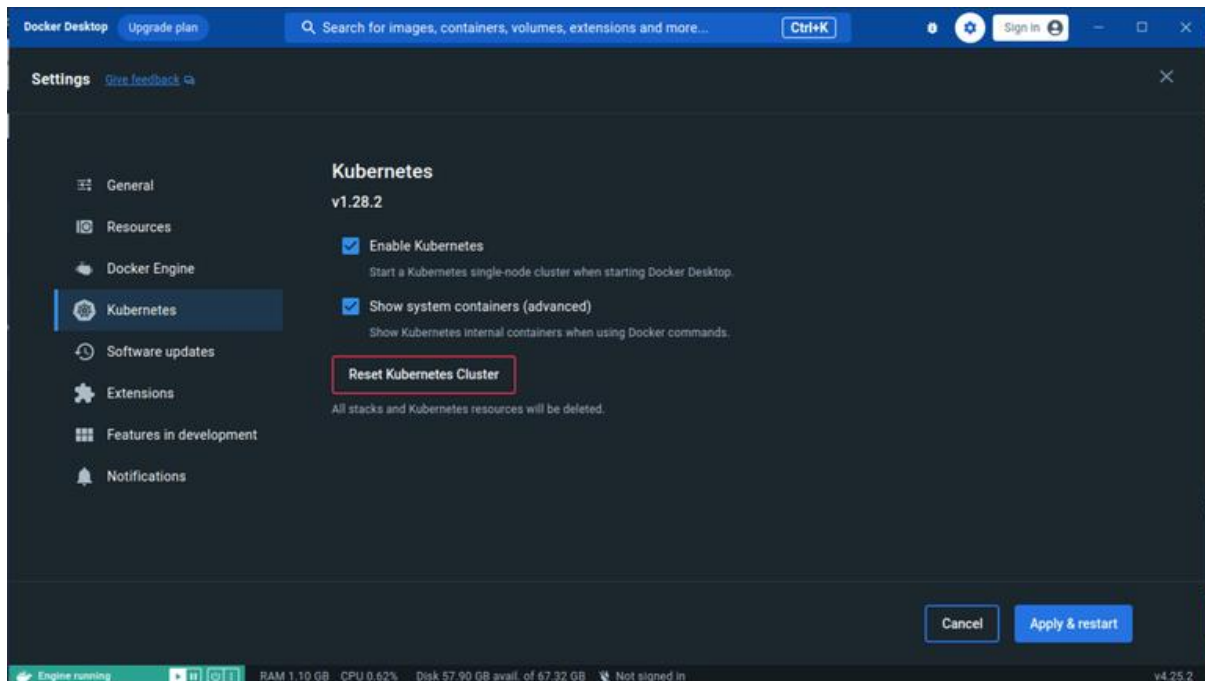
Deploying Kubernetes Clusters with Docker Desktop

Step 1: Verifying Docker Desktop Installation, by putting following code in command prompt

To ensure Docker Desktop is installed correctly, open a terminal or command prompt and run.

docker version

Step 2: Enabling Kubernetes In Docker Desktop



Step 3: Verifying Kubernetes Installation

Check the Kubernetes installation by running the **'Kubernetes version'**.

kubectl version

```
Client Version: v1.32.2
Kustomize Version: v5.5.0
Server Version: v1.32.2
```

Step 4: Deploying A Sample Application

Create a YAML file, **'sample-deployment.yaml'**, with the following contents:

```
! sample-deployment.yaml X Dockerfile .dockerignore require
docker_stuff > ! sample-deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: sample-app
5  spec:
6    replicas: 3
7    selector:
8      matchLabels:
9        app: sample-app
10   template:
11     metadata:
12       labels:
13         app: sample-app
14     spec:
15       containers:
16         - name: sample-container
17           image: nginx:latest
18           ports:
19             - containerPort: 80
20
```

step 4: Apply the deployment to the Kubernetes cluster:

```
kubectl apply -f sample-deployment.yaml
```

```
cosmos@cosmos:/mnt/devesh/code/test$ kubectl apply -f sample-deployment.yml
deployment.apps/sample-app created
```

Step 5: Checking Deployment Status

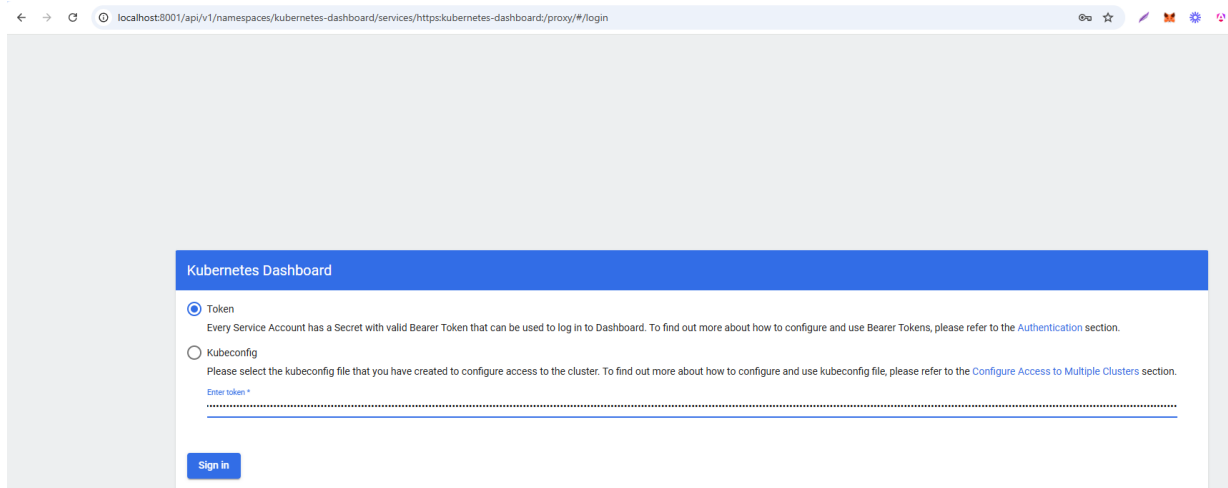
```
kubectl get pods
```

```
cosmos@cosmos:/mnt/devesh/code/test$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
sample-app-5595794bc9-5sn6t        1/1     Running   0           73s
sample-app-5595794bc9-t9669        1/1     Running   0           73s
sample-app-5595794bc9-xmz6c        1/1     Running   0           73s
```

Step 6: Accessing The Deployed Application

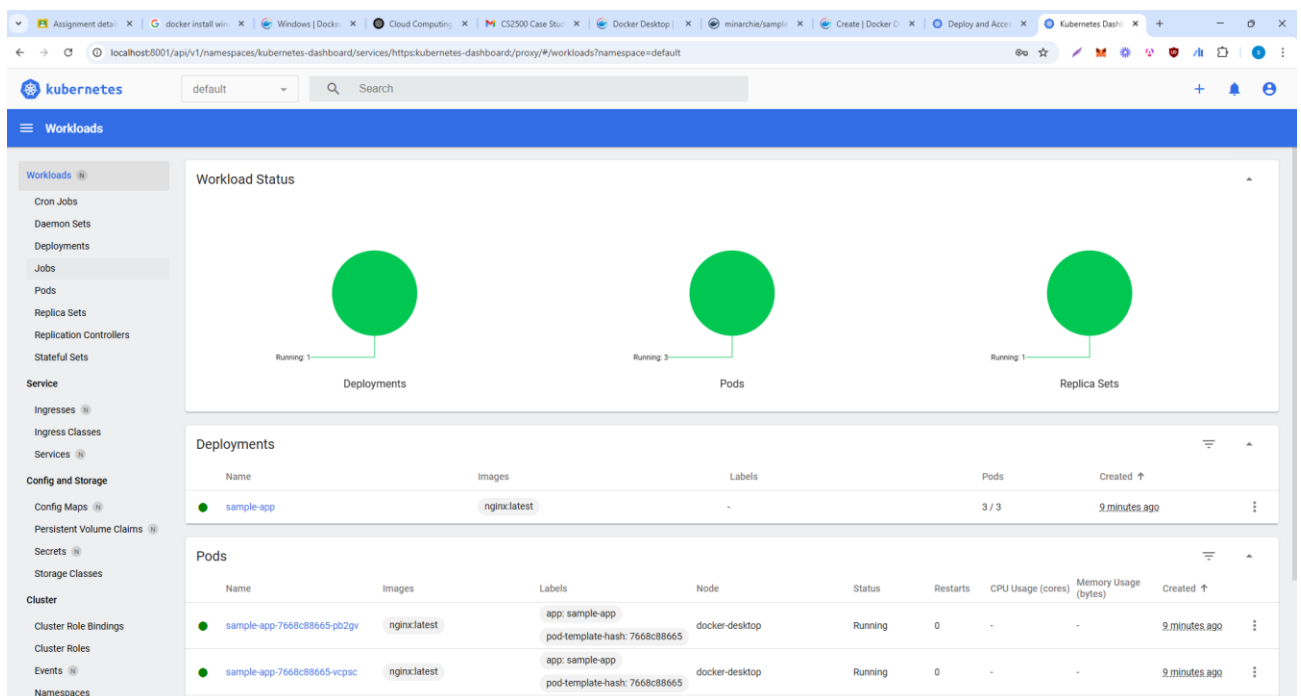
Retrieve the external IP address of the deployed application:

```
kubectl get svc
```

kubectl dashboard

The dashboard provides a graphical interface for managing and visualizing your clusters.



Kubernetes Context Switching

Switch between different Kubernetes contexts:

```
kubectl config get-contexts
```

```
kubectl config use-context <context-name>
```

This is useful when working with multiple clusters.


```
cosmos@cosmos:~$ kubectl config get-contexts
CURRENT   NAME             CLUSTER           AUTHINFO           NAMESPACE
* docker-desktop  docker-desktop    docker-desktop     docker-desktop
minikube   minikube          minikube           minikube           default
cosmos@cosmos:~$ kubectl config use-context docker-desktop
Switched to context "docker-desktop".
```

```
kubectl get nodes
cosmos@cosmos: /mnt/devesh/code/test$ kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
docker-desktop   Ready     control-plane  8m16s  v1.28.2
```

Ensure that `kubectl` points to the Docker Desktop context:

```
kubectl config use-context docker-desktop
```

Expected output shall be similar to: **Switched to context “docker-desktop”**.

6.PUPPET

What is Puppet?

Puppet is a configuration management tool used for automating system administration.

Example Puppet Manifest

```
file { ['/tmp/hello.txt']:
  ensure => present,
  content => "Hello, Puppet!",
}
```

7.CHEF

What is Chef?

Chef is a configuration management tool used to automate infrastructure setup.

Example Chef Recipe

```
file '/tmp/hello.txt' do
  content 'Hello, Chef!'
  action :create
end
```

8.ANSIBLE

Ansible is used for setting up environments, deploying applications, managing configurations.

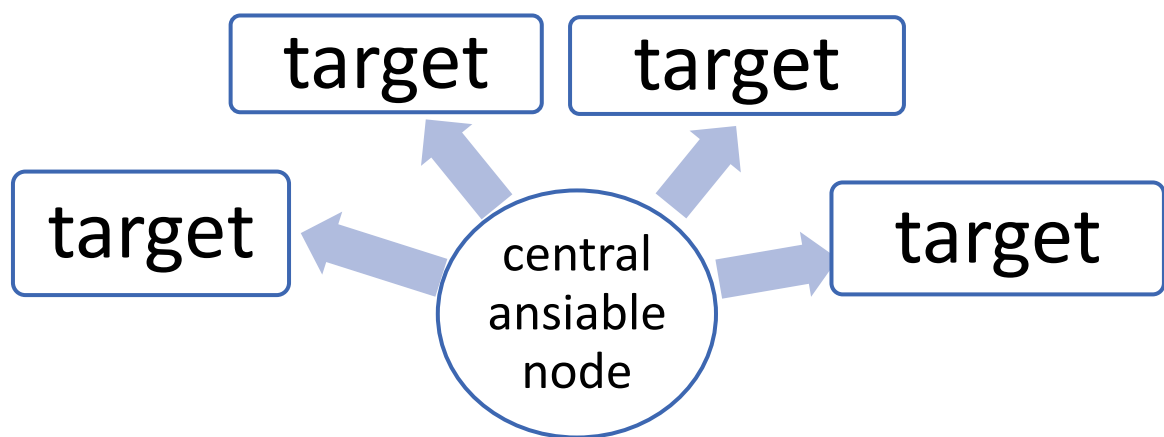
Ansible is a tool that helps you automate tasks on multiple computers at once.

Ansible is more efficient and less time in 4 different ways:

- Execute tasks from your own machine
- Configuration/Installation/Deployment steps in a single YAML file
- Re-use the same file Multiple times and for different environment.
- More reliable and less likely for errors

Ansible playbook: An Ansible playbook is a file written in YAML format that defines a set of tasks to be executed on remote servers or machines. These tasks typically involve like installing software, configuring settings, or deploying applications.

Ansible work with Modules Like small program.



9. JENKINS

What is Jenkins?

Jenkins is an open-source automation tool for CI/CD. It allows developers to build, test, and deploy code efficiently.

Key Features

- Automates software development processes
- Supports integration with multiple tools
- Provides pipeline-as-code functionality

Jenkins Pipeline Script

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'make build'
      }
    }
    stage('Test') {
      steps {
        sh 'make test'
      }
    }
    stage('Deploy') {
      steps {
        sh 'make deploy'
      }
    }
  }
}
```

Jenkins Search (#+K) Admin log out

Dashboard > creating-a-pipeline-in-blue-ocean > First-pull-branch

Status

- Changes
- Build Now
- View Configuration
- Full Stage View
- Open Blue Ocean
- GitHub
- Pipeline Syntax

Build History trend

Filter builds...

- #4 Aug 8, 2022, 2:14 PM
- #3 Aug 4, 2022, 8:49 PM
- #2 Aug 3, 2022, 2:18 PM

Branch First-pull-branch

Full project name: creating-a-pipeline-in-blue-ocean/First-pull-branch

Stage View

Average stage times:
(Average full run time: ~1h 29min)

	Declarative: Checkout SCM	Build	Test	Test	error	Deliver
#4 Aug 08 14:14 No Changes	1s	36s	36ms	1s	70ms (paused for 1h 24min)	5s (paused for 4min 24s)
#3 Aug 04 20:49 No Changes	472ms	31s	37ms	1s	249ms (paused for 3d 17h) aborted	69ms aborted
#2 Aug 03 14:18 No Changes	7min 27s					

Permalinks

10. RPA

What is RPA?

RPA automates repetitive tasks using software bots. It is used in business processes like data entry and report generation.

Popular RPA Tools

- UiPath
- Blue Prism
- Automation Anywhere

11. SELENIUM

What is Selenium?

Selenium is an open-source tool for automating web browser testing.

Example Selenium Test

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://www.google.com")
print(driver.title)
driver.quit()
```

Summary:

The document provides a comprehensive overview of various aspects within software engineering and DevOps practices. It begins with an introduction to classical software engineering methodologies such as Waterfall, Spiral, and V-Model, contrasting them with Agile methodologies like Scrum, Kanban, and Extreme Programming (XP). Each approach is explained with its principles, benefits, and typical use cases in software development.

The section on Operating Systems outlines the fundamental roles of OS in managing computer hardware and software resources, covering areas such as file, memory, device, security, and process management. It emphasizes the OS's crucial role as an intermediary between users and hardware, ensuring efficient program execution and user interaction.

The document also delves into various DevOps tools essential for modern software development, including Pylint for code quality, PyUnit for unit testing, Git/GitHub for version control and collaboration, Docker/Docker Hub for containerization, Kubernetes for container orchestration, and Ansible+ other tools for automation and configuration management. Each tool is explained in detail, from installation procedures to practical usage examples, highlighting their significance in streamlining development processes and enhancing team productivity.

In summary, the document serves as a comprehensive guide to both traditional and modern software engineering practices, alongside essential DevOps tools that facilitate collaborative, automated, and efficient software development lifecycle management.